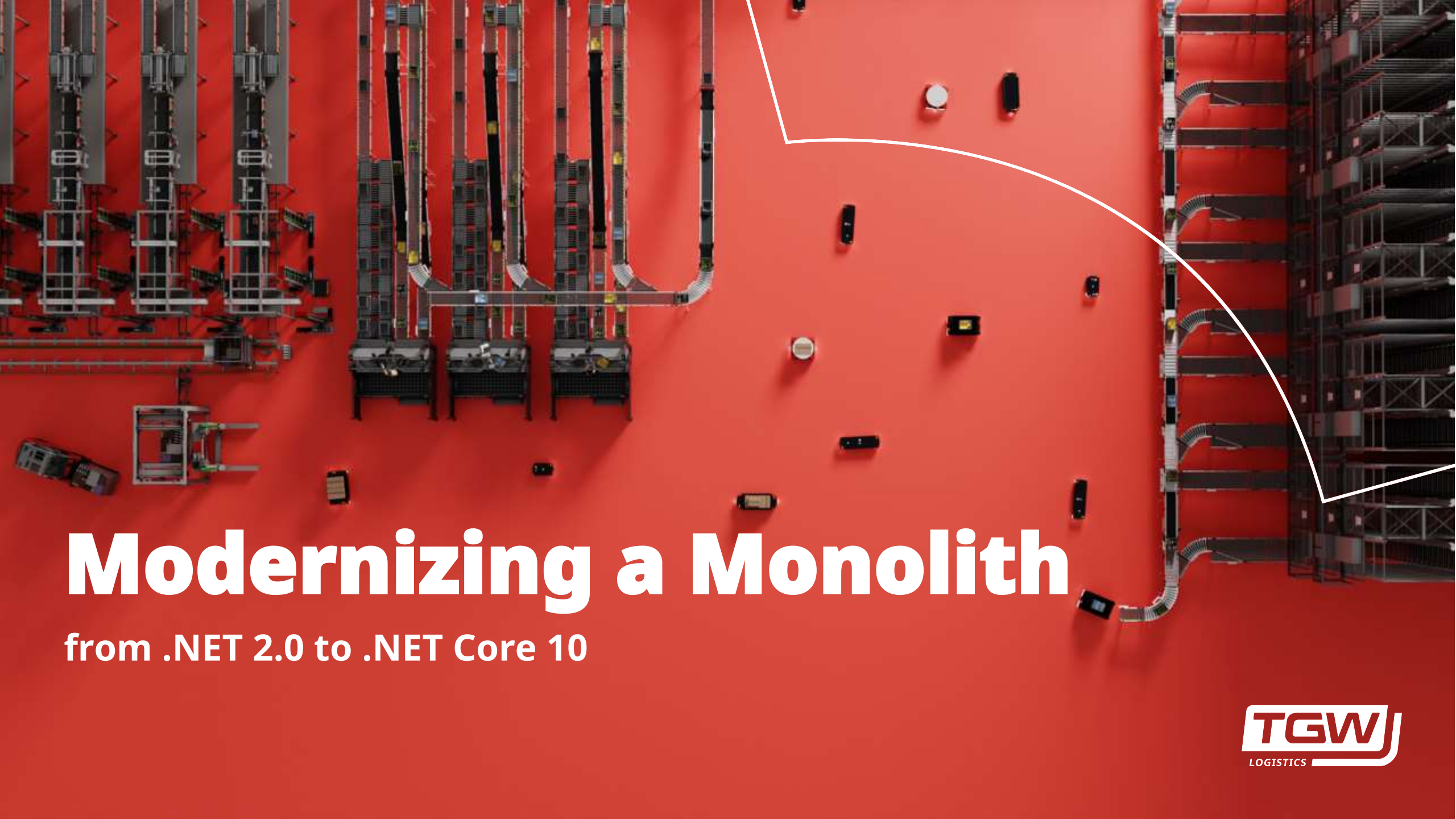




Modernizing a Monolith

from .NET 2.0 to .NET Core 10

About me

Klaus Linzner

- Software Architect @ TGW
- Strong Background in .NET – working on base .NET libs for years, some C# talks at conferences
- Couple years ago heading more into build & delivery mechanisms
 - Lead migration to git and azure devops
- Delivery mechanisms and builds were extended to software delivery to site
 - Lead on internal project ADP „Advanced Delivery Platform“ – the project that introduced containerization at scale
 - Part of this was the dotnet core migration



What (not) to expect

- You will not see a „best practices“
 - try to describe the environment we're working in,
 - its challenges,
 - how we try to mitigate / solve them
- If any questions come up please just raise them
 - If small enough I'll try to answer inline, worst case we need to postpone to Q&A / discussion at the end
- I'm very interested in any recommendations, remarks or ideas
 - We definitely left room for improvement



Outlook

- Bit of context in which environment we're working (it's „special“)
- .NET Timeline – were we were 2015
- Leading up to migration (~2017-2020)
- NetCore part (~2022-2024)
- Wrap up



An aerial photograph of a large industrial warehouse complex. The warehouse has a long, low profile with multiple loading docks. Numerous semi-trailers are parked in neat rows along the docks. In the foreground, there are large stacks of goods, including white palletized units and red crates. The facility is surrounded by greenery and trees. A large red graphic element is overlaid on the right side of the image.

WHAT WE'RE DOING



warehouses

**millions of
crates**

**hundreds of
devices**

hundreds of times



zalando PUMA®

PIC
NIC

TGW
LOGISTICS

coop

INTERSPORT

playmobil®

th.mann ESPRIT

SKECHERS

DECATHLON

GAP

ICA



STRAUSS

KÄRCHER

mayoral



VICTORIA'S SECRET

MANGO

hachette
UK

what we're doing

a single device can be...

a storage rack



what we're doing

a single device can be...

a shuttle block



what we're doing

a single device can be...

a (part of a) conveyor



what we're doing

a single device can be...

an AMR / AGV



what we're doing

a single device can be...

a palletizer



what we're doing

a single device can be...

a pick station



what we're doing

Depending on customer needs we deliver everything from ground up.

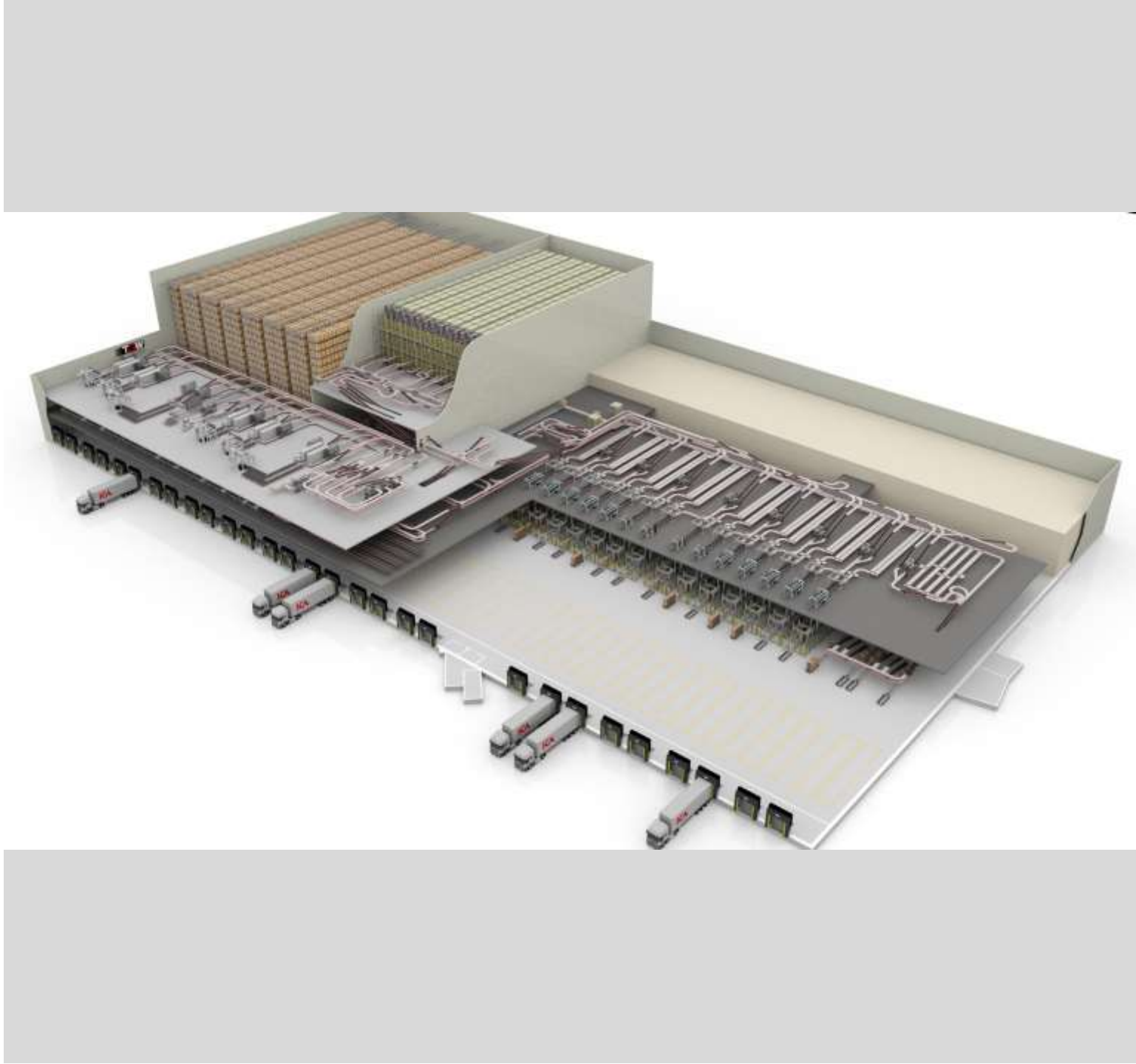
Each site is scaled / designed to customer requirements (number of transports, storage locations) which determines compute resources.



what we're doing

While standardization is a big factor,
many of those components are tailored to customer needs.

This means customer specific
application code, database code,...



what we're doing

The workload we're running is usually rather static.

Peak times usually black friday to christmas.

(can go so far: last chance of deployment for the year is middle of october – next time middle of january)



what we're doing

Many of those components are time-critical.

With those time constraints, our software usually needs to run in the customers datacenter(s).

This means within the customers infrastructure.



How we work

- From development point of view: There's a split within TGW
 - Central PD (Product Development, AT&DE): Creates base products
 - Realization Units (AT,CN,DE,ES,FR,IT,UK,US): Taking those base products, customizing/extending them and fulfilling customer projects
- Order of magnitude in 2026:
 - 100 developers in PD
 - 300-400 developers in Realization





TFVC

WPF

Silverlight™

Road to hell
is paved with
good intentions.

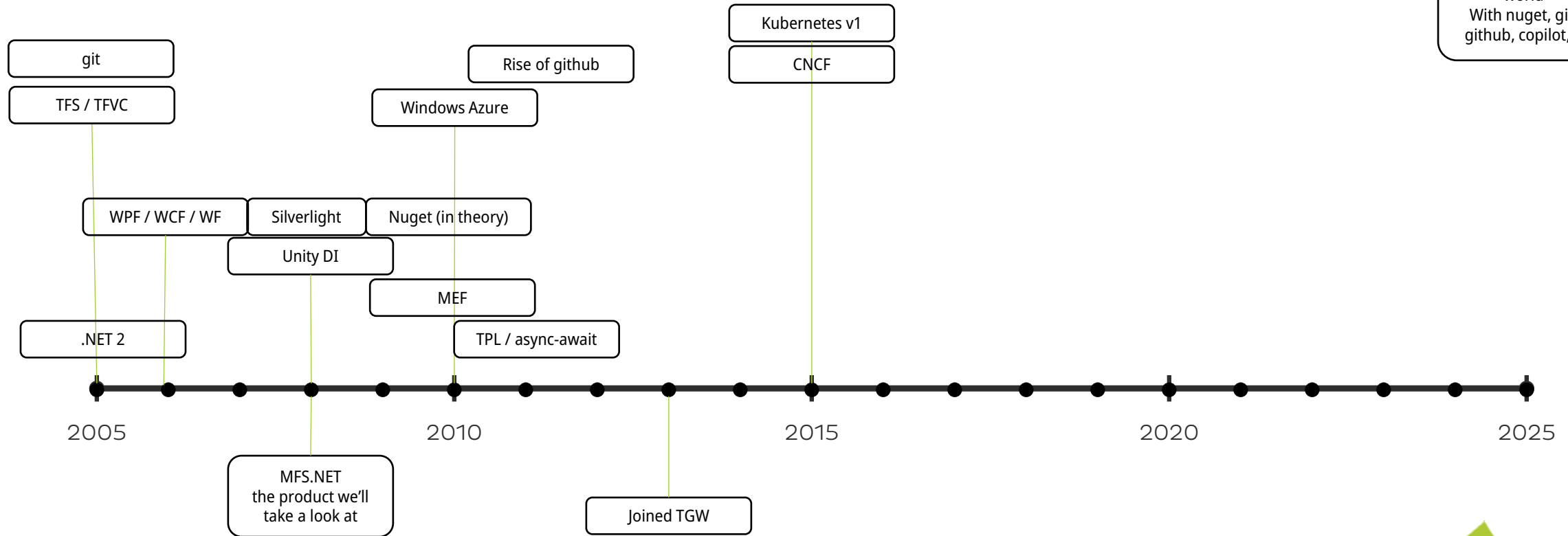
WCF

MEF

.NET centered timeline



Our shiny .NET 10
world
With nuget, git,
github, copilot,...

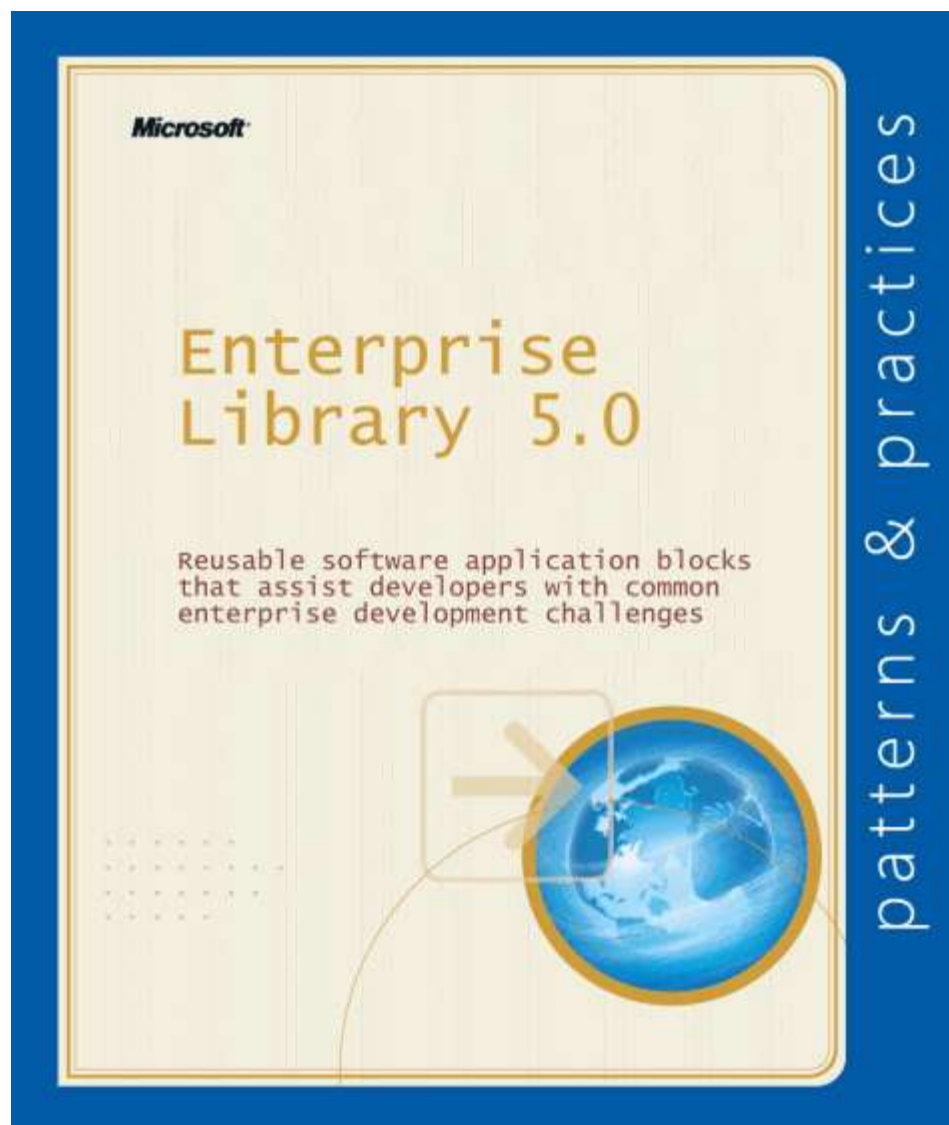


<200
(Revenue)

<300

~450





- Two main products
 - Full Microsoft stack: .NET, TFVC, MEF, WCF, WF
 - Builds are running on some kind of home grown infrastructure (self managed VMs)
 - Two main products share same internal libraries
- PD
 - Creates products
 - Compiles them, creates MSI installers
- REA
 - Extend products
 - Have their own solutions with customizations
 - Place libraries in magic places – they are loaded from there and Dependency Injection runs

- Overly dramatic:
 - „We don't want to deal with your internal layerings – hide your complexity and provide us functionality so we only have to reference one assembly“
- kind of the opposite of IOC (Inversion of control) – but ... It was the style at the time
- It worked – but we had to suffer consequences
 - Binary compatibility
 - Explicit Assembly Loading
 - Assembly Redirect Injecting
 - App Domain handling / interaction
 - Pretty much – no compiler help on those topics
 - For realization: They had copy/paste templates that were passed from team to team – and that had to be manually updated when we brought significant changes

Add Azure Build for PR validation

Completed !184 Klaus Linzner proposes to merge [User/LZK/AzureCheckin](#) into [master](#) All comments resolved

Overview Files Updates Commits Conflicts

All Changes

Filter

1 changed file

MFC

Builds

azure-checkin.yml

azure-checkin.yml +37
/Builds/azure-checkin.yml

```
1 + # .NET Desktop
2 + # Build and run tests for .NET Desktop
3 + # Add steps that publish symbols, s
4 + # https://docs.microsoft.com/azure
5 +
6 + trigger:
7 +   branches:
8 +     include:
9 +       - master
10 +       - release/*
11 +       - feature/*
12 +
13 + pool:
14 +   vmImage: 'windows-latest'
15 +
16 + variables:
17 +   solution: 'Products/Commander.sln'
18 +   buildPlatform: 'Any CPU'
19 +   buildConfiguration: 'Release'
20 +
21 + steps:
22 + - checkout: self
23 +   lfs: true
24 +   fetchDepth: 1
25 +
26 + - task: NuGetToolInstaller@1
27 +
28 + - task: NuGetCommand@2
29 +   inputs:
30 +     restoreSolution: '$(solution)'
31 +
32 + - task: VSBUILD@1
33 +   inputs:
34 +     solution: '$(solution)'
35 +     platform: '$(buildPlatform)'
36 +     configuration: '$(buildConfiguration)'
37 +
```

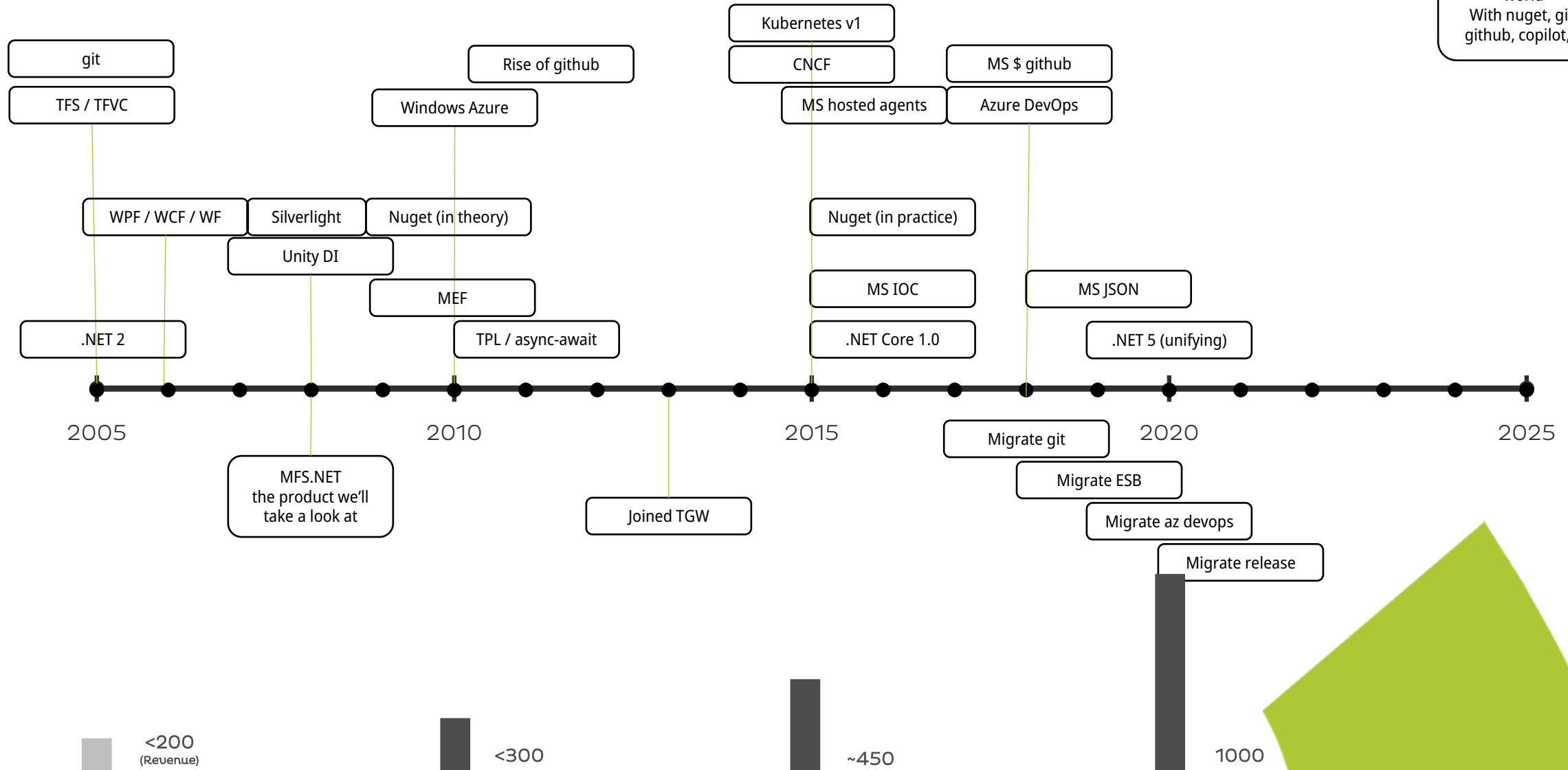
**LONG BEFORE
THINKING
ABOUT
.NETCORE:**

**LEADING UP TO
THE MIGRATION**

~ 2017-2020

.NET centered timeline

Our shiny .NET 10
world
With nuget, git,
github, copilot,...



Migrate TFVC -> GIT

■ One of first important steps was migrating from TFVC to git.

- Including the full history
- Branching model and internals between TFVC are vastly different (is a branch an actual object showing up in history,...)
- Actual migration took couple of weekends, repo per repo (in itself: repo in TFVC vs repo in git are vastly different scope)
- Figuring out the exact commands so we can migrate everything was a PITA (branching in TFVC is an entirely different beast – and was maybe misused on our side)

■ <https://github.com/git-tfs/git-tfs>

```
Fetching from dependent TFS remote 'Systems/Release/v3.4' ...
C38401 = eb09266b2ee646344511fb2b38697f30a40579ed
C38408 = 12402d0b189bc359b097d1216db8dfece3c68818
C38475 = ee9cd9a896ccaebe14a4b204a0d2d5aa0f034e65
C38525 = e0fe2bf381dce76b3405afb9667f5b5ab67b4409
C38550 = 6d81be39388fca99940d5744e47680d987c71575
C38551 = 2886166b428c1ec5965f32c4e939805ffd33c2e3
C38643 = 36e43057bb64bde3d3e219cc1ab708ca453c31d0
```

warning: Some Tfs branches could not have been initialized:

```
- $/Frameworks/Systems/Release/v3.2
- $/Frameworks/Systems/Release/v3.3
- $/Frameworks/Systems/Dev/LZK-NetStandard
- $/Frameworks/Systems/Release/v3.4
- $/Frameworks/Systems/Release/v4.0
- $/Frameworks/Systems/Release/v4.Latest
```

Please report this case to the git-tfs developers! (report here : <https://github.com/git-tfs/git-tfs/issues/461>)

warning: Some Tfs branches could not have been initialized or entirely fetched due to errors:

```
- $/Frameworks/Systems/Release/v3.2
=>error:error: Couldn't fetch parent branch

- $/Frameworks/Systems/Release/v3.3
=>error:error: Couldn't fetch parent branch

- $/Frameworks/Systems/Dev/LZK-NetStandard
=>error:error: Couldn't fetch parent branch

- $/Frameworks/Systems/Release/v3.4
=>error:error: Couldn't fetch parent branch

- $/Frameworks/Systems/Release/v4.0
=>error:error: Couldn't fetch parent branch

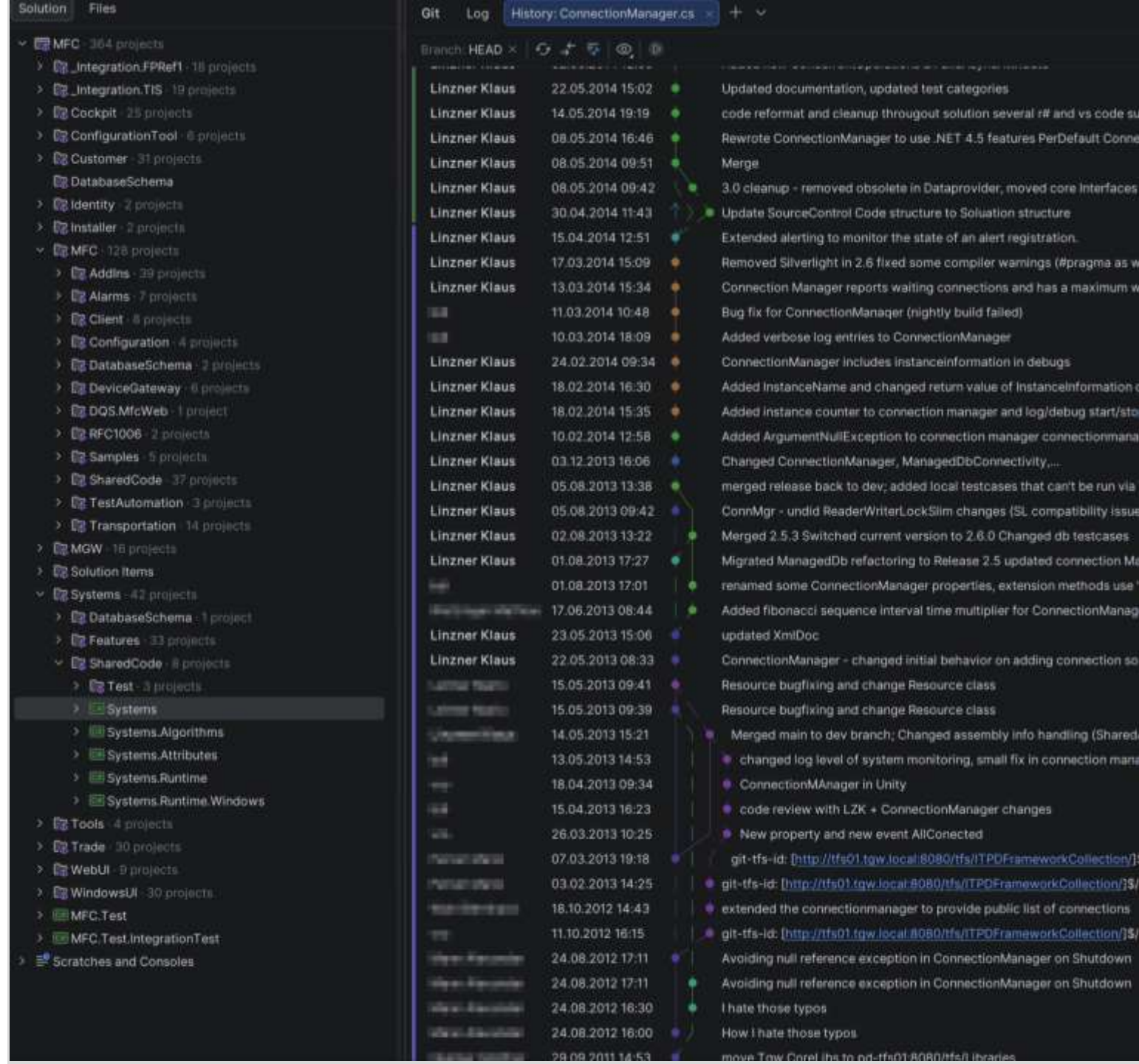
- $/Frameworks/Systems/Release/v4.Latest
=>error:error: Couldn't fetch parent branch
```

Please report this case to the git-tfs developers! (report here : <https://github.com/git-tfs/git-tfs/issues>)

```
D:\Dev\SystemGit>
```

Migrate Monorepo

- Historically we had many different repos with different release cycles
- Made changes / refactoring / integration / releasing a ton of effort
- After we had everything in git we could merge into a single repo and have a single branch for any refactorings – and complete the PR once **everything** was updated
- Still had the complete code history
- Full disclosure: 7 years later we moved part of it out again.
 - In Hindsight: Move to mono-repo was the right thing to do
 - Moving part of it out as well



Migrate Messaging

Our old Enterprise Service Bus ...

- relied on MSMQ / Windows
- provided tons of features (that we did not use)
- Forced us to work around several issues (not accessible from windows clients,...)

Self induced complexity

- Initially planned to hide underlying complexity by providing a base class the deals with lots of internals
- Provided a base implementation (by us – not their fault) – requiring subclassing mechanism
- Led to messaging system internals bleeding through stack

```
BaseNeuronSenderReceiver.cs
1  > #region Copyright...
18
11  > usings
29
30  namespace Tgw.Systems.Communication.Esb
31  {
32  >  /// Represents an abstract base class for sending and receiving messages using Neuron Enterprise Se
    35 usages  34 inheritors  Linzner Klaus +4  1+16 extension methods
41  >  public abstract partial class BaseNeuronSenderReceiver : EsbSenderReceiverSlim
42  >  {
43  >  Constants and Fields
49
50  >  Events
58
    2 usages  Linzner Klaus +1
59  >  protected BaseNeuronSenderReceiver(string partyName)
60  >  : this(NeuronConfigurationSupport.GetConfiguredNeuronConfiguration(partyName)){...}
63
64  >  /// Creates a new instance using the given ESB configuration. ...
    16 usages  bdi +1
68  >  protected BaseNeuronSenderReceiver(NeuronConfiguration neuronConfiguration)
69  >  : this(neuronConfiguration, useWcf: false, wcfCallbackUri: null){...}
72
73  >  /// Creates a new instance and uses WCF as communication platform. For service agents used in a
    1 usage  bdi +1
82  >  protected BaseNeuronSenderReceiver(NeuronConfiguration neuronConfiguration, Uri wcfCallbackUri)
83  >  : this(neuronConfiguration, useWcf: true, wcfCallbackUri){...}
86
87  >  /// Creates a new instance, using WCF as communication platform. For service agents used in a U
    9 usages  Linzner Klaus +1
96  >  protected BaseNeuronSenderReceiver(NeuronConfiguration neuronConfiguration, bool useWcf, Uri wcf
97  >  : base(neuronConfiguration,
98  >  useWcf,
99  >  useWcf
100  >  ? CreateCallbackUriIfNull(wcfCallbackUri, neuronConfiguration)
101  >  : wcfCallbackUri){...}
109
```

Migrate Messaging

- Migrated to Redis (Redis Streams is awesome for messaging) first, later on replaced with RabbitMq
- Lessons learned: Underlying messaging should not be visible to applications anymore
- We brought something that we named ProxyMessaging: Applications only declare the functional interface...

```
public interface IMaterialFlowControllerServer
{
    /// Occurs when a LoadCarrierRequest has completed. ...
    event PersistentAsyncCompletedEventHandler<CreateTransportResponseWms> CreateTransportCompleted;

    /// Called to initiate sending a LoadCarrier to a destination. ...
    [FiresEvent(nameof(CreateTransportCompleted))]
    void CreateTransport(Correlated<CreateTransportRequestWms> loadCarrierRequest);

    event PersistentAsyncCompletedEventHandler<CreateSequenceTransportResponseWms> CreateSequenceTransportCompleted;

    /// Called to initiate sending a LoadCarrier to a destination. ...
    [FiresEvent(nameof(CreateSequenceTransportCompleted))]
    void CreateSequenceTransport(Correlated<CreateSequenceTransportRequestWms> loadCarrierRequest);

    /// Raised when a sequence group has been completed successfully or failed with an error. ...
    event EventHandler<SequenceGroupCompletedEventArgs> SequenceGroupCompleted;

    /// Occurs when a Load Carrier is identified. ...
    event EventHandler<LoadCarrierAppearedInfoWms> LoadCarrierAppeared;

    /// Raised when internal MFC transports have been created (e.g. blocking load carriers). ...
    event EventHandler<InternalTransportsCreatedEventArgs> InternalTransportsCreated;

    /// Raised when a source load carrier of a sequence group reaches a configurable address. ...
    event EventHandler<TargetRequestedEventArgs> TargetRequested;

    /// Raised after a load carrier in transport has left the source address ...
    event EventHandler<LoadCarrierInTransportEventArgs> LoadCarrierInTransport;

    /// Occurs when the progress of processing a Load Carrier Request changes. ...
    event EventHandler<TransportProgressChangedEventArgs> TransportProgressChanged;
```

Migrate Messaging

- ...and create an instance of the messaging abstraction, rest is auto-wired
- There was some abstraction bleeding – but in the end little changes that needed to be made in switch from Redis to RabbitMq
- First implementation was still a bit clumsy, but current implementation IMO is rather nice to look at (we'll see this later on)

AddIn.cs x

```
public class MaterialFlowAddIn : IMFCAddin
{
    public void Initialize()
    {
        log.Info($"Loading {nameof(MaterialFlowAddIn)}...");

        string systemKey = ConfigurationSupport.GetSpecificSystemKey();

        MaterialFlowAdapter mfcAdapter = new MaterialFlowAdapter();
        Container.BuildUp(mfcAdapter);
        Container.RegisterInstance<IMaterialFlowControllerServer>(mfcAdapter);
        RegisterLifecycleService(mfcAdapter);

        //this service active as a service
        IStreamsConnectionEnvironment factory = StreamsConnectionEnvironment.Default;
        IStreamsConnection controllerConnection =
            factory.AddPartyConnection(factory.PartyIds.MFCServer(systemKey));
        IServiceFacade<IMaterialFlowControllerServer> controllerFacade =
            controllerConnection.CreateServiceFacade<IMaterialFlowControllerServer>(mfcAdapter);

        //this service needing to connect to another service
        IStreamsConnectionEnvironment sisFactory = StreamsConnectionEnvironment.Default;
        IStreamsConnection sisServiceConnection =
            sisFactory.AddPartyConnection(sisFactory.PartyIds.HostWMCallback(systemKey));
        IWarehouseManagementOperations sisServiceAgent =
            sisServiceConnection.CreateProxyFor<IWarehouseManagementOperations>(sisFactory.PartyIds.HostWMCallback(systemKey));

        //Add connection to managers for lifecycle management
        ConnectionManager.AddConnectionAsync(sisServiceConnection).WaitAndUnwrap();
        ReaderManager.AttachReaderAsync(sisServiceConnection).WaitAndUnwrap();

        Container.RegisterInstance(controllerFacade);
        ConnectionManager.AddConnectionAsync(controllerConnection).WaitAndUnwrap();
        ReaderManager.AttachReaderAsync(controllerConnection).WaitAndUnwrap();
    }
}
```

Migrate DevOps Builds

■ Until 2020 mostly running on custom build infrastructure

■ Upside

- It started of when the market had no good offer
- It had lots of special features like integration into DBMS to reverse engineer, support for subversion/svn/...

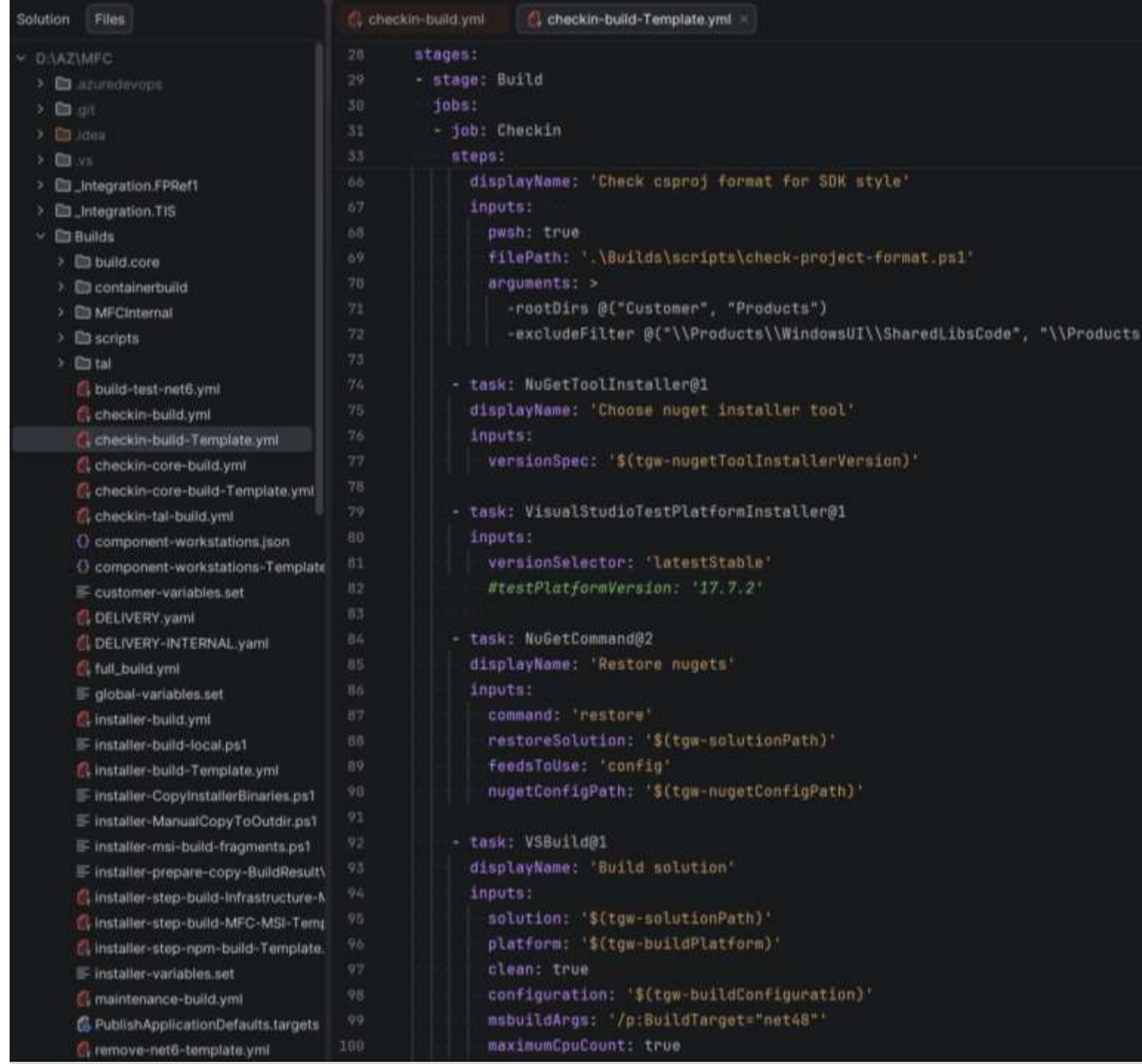
■ Downsides:

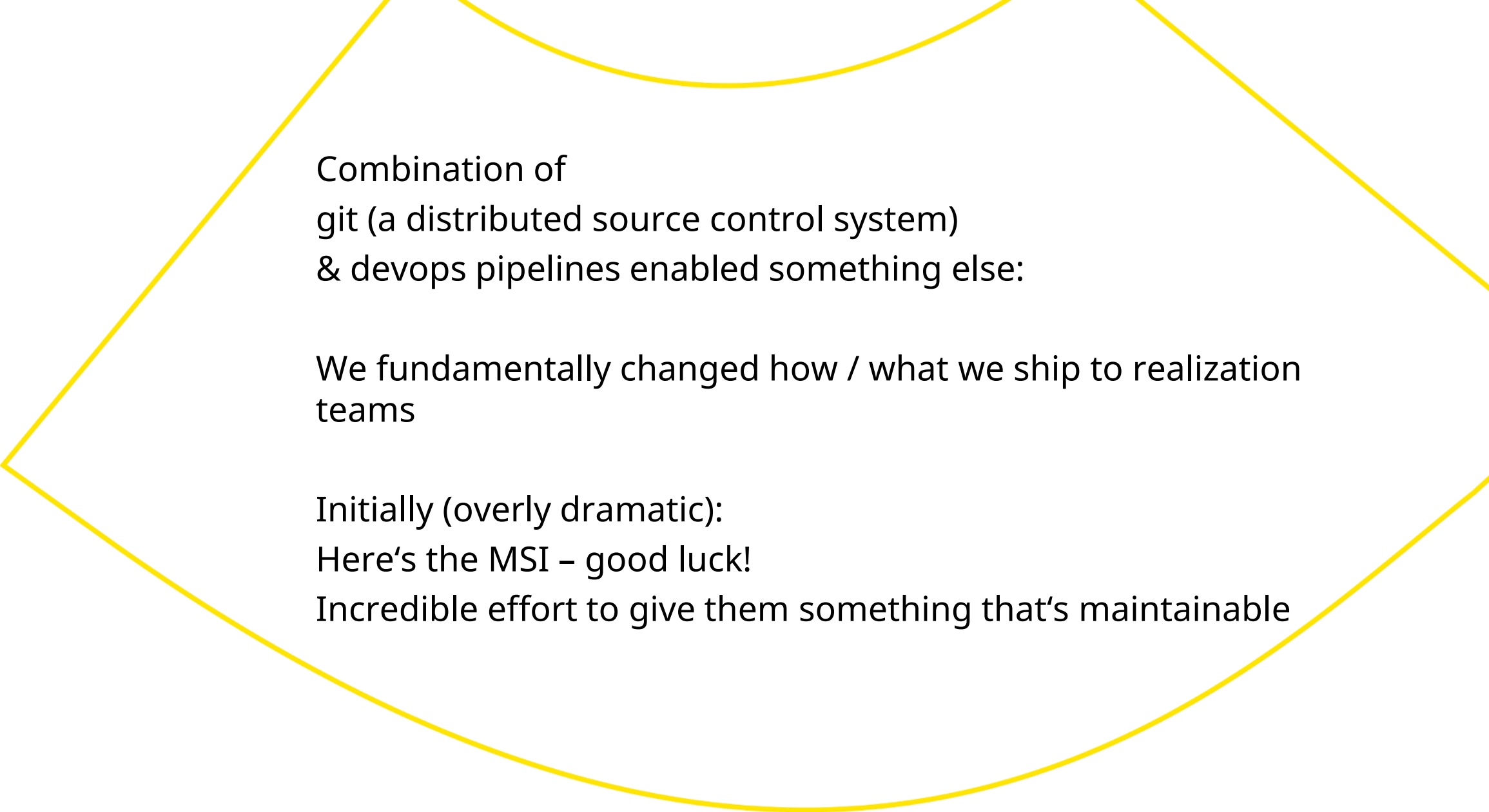
- Features like git support had to be implemented
- No dynamic / horizontal scaling
- Scaling based on configuration
- Huge effort in (manual) maintenance of build infrastructure (20-30 servers)
- No PR integration

```
28  stages:
29    - stage: Build
30      jobs:
31        - job: Checkin
32          steps:
33            - task: NuGetToolInstaller@1
34              displayName: 'Check csproj format for SDK style'
35              inputs:
36                pwsh: true
37                filePath: '.\Builds\scripts\check-project-format.ps1'
38                arguments: >
39                  -rootDirs @("Customer", "Products")
40                  -excludeFilter @(".*\Products\WindowsUI\SharedLibsCode", ".*\Products
41
42            - task: NuGetToolInstaller@1
43              displayName: 'Choose nuget installer tool'
44              inputs:
45                versionSpec: '$(tgw-nugetToolInstallerVersion)'
46
47            - task: VisualStudioTestPlatformInstaller@1
48              inputs:
49                versionSelector: 'latestStable'
50                #testPlatformVersion: '17.7.2'
51
52            - task: NuGetCommand@2
53              displayName: 'Restore nugets'
54              inputs:
55                command: 'restore'
56                restoreSolution: '$(tgw-solutionPath)'
57                feedsToUse: 'config'
58                nugetConfigPath: '$(tgw-nugetConfigPath)'
59
60            - task: VSBUILD@1
61              displayName: 'Build solution'
62              inputs:
63                solution: '$(tgw-solutionPath)'
64                platform: '$(tgw-buildPlatform)'
65                clean: true
66                configuration: '$(tgw-buildConfiguration)'
67                msbuildArgs: '/p:BuildTarget="net48"'
68                maximumCpuCount: true
```

Migrate DevOps Builds

- Not all features from our custom build infrastructure were available in devops
- Moved on nevertheless
 - Maintenance effort dropped massively
 - Automation of various aspects increased massively





Combination of
git (a distributed source control system)
& devops pipelines enabled something else:

We fundamentally changed how / what we ship to realization teams

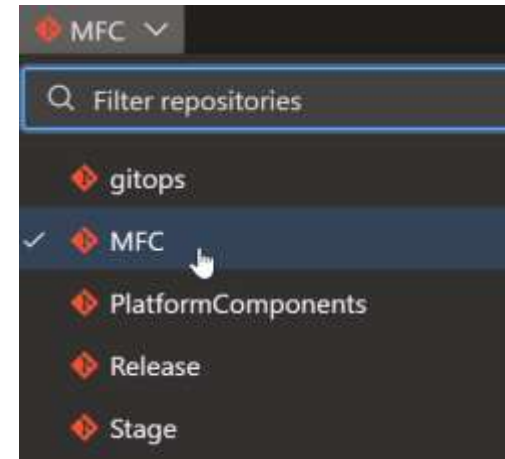
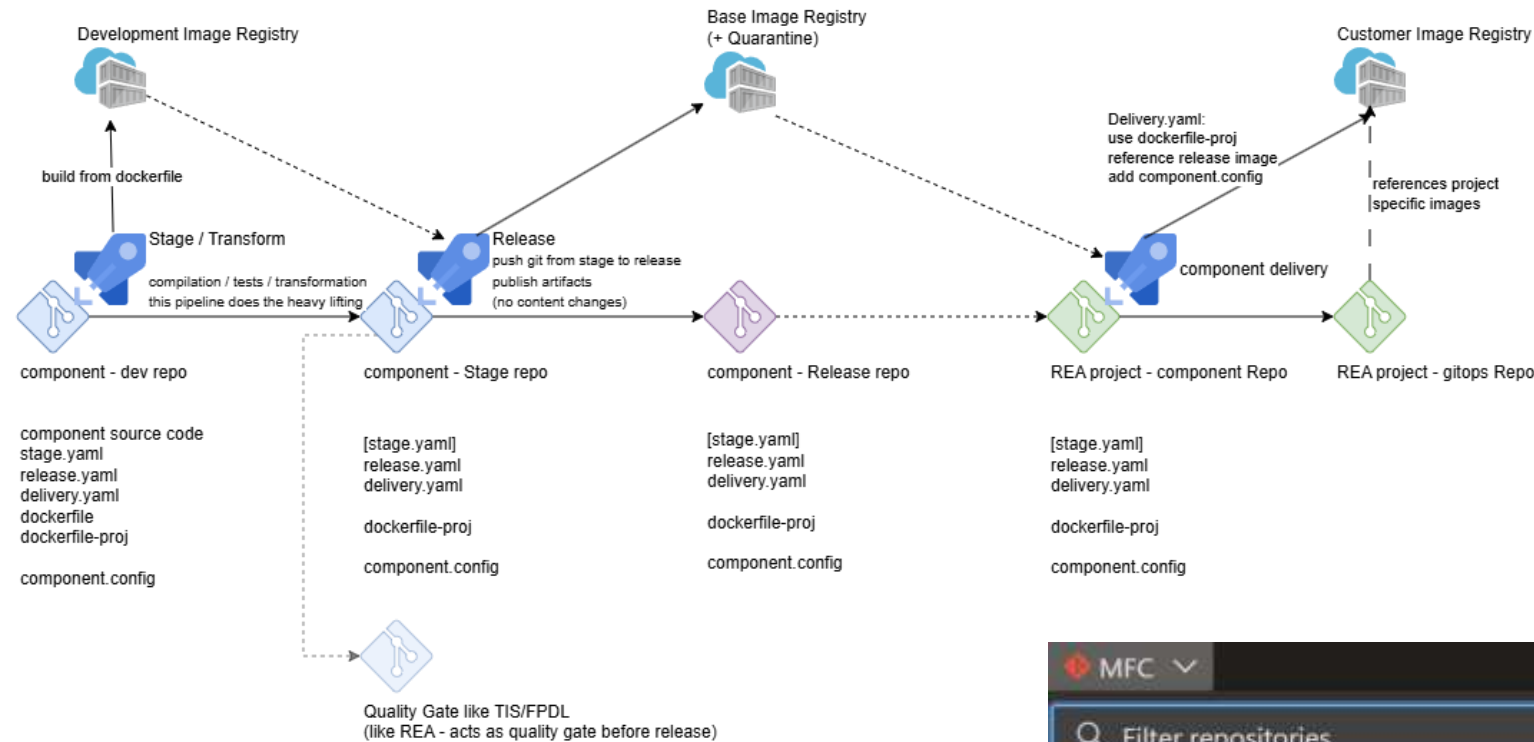
Initially (overly dramatic):
Here's the MSI – good luck!
Incredible effort to give them something that's maintainable

Migrate Release Mechanism

Realization teams need

- Base product
- A customer solution to get started
- Builds

We now had everything API accessible, did know their technical capabilities and infrastructure...
...and could automate much, much more...



Migrate Release Mechanism

- Usually this would not be a topic for the .NET meetup, but...
- it allowed us to bring changes and migration to our customers
- With a rather simple „git pull upstream“ they could remain on net48, but could also kick of the net60 pipeline that just worked

Update to v8.3.0.117960...

897898b4 Release Transformation committed Aug 30, 2024 Release/MFC/v8.3

Files Details

Parent 1 → This commit Filter Customer.DeviceGateway.Server.csproj -29 +28
/Customer/MFC/Source/DeviceGateway.Server/Customer.DeviceGateway.Server.csproj

Release

- Builds
- ConfigFiles
- Customer
 - Client/Source/MFC.Client
 - Cockpit/Source
 - ConfigurationTool/Source
 - MFC
 - Source
 - Alarms.Server
 - DeviceGateway.Domain
 - DeviceGateway.Server
 - App.config
 - CustomDeviceGatewayApplicationLoade...
 - Customer.DeviceGateway.Server.cs... 1
 - Program.cs
 - TGW.Systems.EnvironmentConfiguration...
 - MFC.ClientConfiguration.Server
 - CustomClientConfigurationServerApplic...
 - Customer.MFC.ClientConfiguration.Serv...
 - Program.cs

```
1 1 <Project Sdk="Microsoft.NET.Sdk">
2 2   <PropertyGroup>
3 3     <OutputType>Exe</OutputType>
4 4     <RootNamespace>TGW.Customer.DeviceGateway.Server</RootNamespace>
5 5     <AssemblyName>TGW.Customer.DeviceGateway.Server</AssemblyName>
6 6     <TargetFramework>net48</TargetFramework>
7 7     <AutoGenerateBindingRedirects>true</AutoGenerateBindingRedirects>
8 8     <AssemblyTitle>Customer.DeviceGateway.Server</AssemblyTitle>
9 9     <Company>TGW Logistics Group GmbH</Company>
10 10    <Product>Customer.DeviceGateway.Server</Product>
11 11    <Description>Core product from TGW Logistics group - Commander Central</Description>
12 12    <Copyright>Copyright © 2020</Copyright>
13 13    <PlatformTarget>AnyCPU</PlatformTarget>
14 14    <Copyright>Copyright ? 2020</Copyright>
15 15    <TargetFrameworks Condition="'$(BuildTarget)' == ''>net48;net6.0</TargetFrameworks>
16 16    <TargetFramework Condition="'$(BuildTarget)' == 'net48'>net48</TargetFramework>
17 17    <TargetFramework Condition="'$(BuildTarget)' == 'net6.0'>net6.0</TargetFramework>
18 18  </PropertyGroup>
19 19   <Import Project="$(SolutionDir)\Builds\PublishApplicationDefaults.targets" Condition="'$(BuildTarget)' == 'net48'>
20 20   <ItemGroup>
21 21     <None Remove="TGW.Systems.EnvironmentConfiguration.xml" />
22 22   </ItemGroup>
23 23   <ItemGroup>
24 24     <Reference Include="Microsoft.ServiceModel.Channels, Version=3.0.0.0, Culture=en-GB, PublicKeyToken=31bf3856ad364e35" />
25 25     <HintPath>..\..\..\..\SharedLibs\Microsoft.ServiceModel.Channels.dll</HintPath>
26 26     <SpecificVersion>False</SpecificVersion>
27 27   </Reference>
28 28   <Reference Include="System.Data.DataSetExtensions" />
29 29   <Reference Include="Microsoft.CSharp" />
30 30   <Reference Include="System.Net.Http" />
31 31   <Reference Include="Tgw.OH.MalfunctionService.WcfMalfunction, Version=5.3.1.0, Culture=en-GB, PublicKeyToken=31bf3856ad364e35" />
32 32   <HintPath>..\..\..\..\SharedLibs\Tgw.OH.MalfunctionService.WcfMalfunction.dll</HintPath>
33 33   <SpecificVersion>False</SpecificVersion>
34 34   </Reference>
35 35   </ItemGroup>
36 36   <ItemGroup>
37 37     <Content Include="..\..\..\..\ConfigFiles\Customer.DGW.config.xml">
38 38       <Link>ConfigFiles\Customer.DGW.config.xml</Link>
39 39     <CopyToOutputDirectory>Always</CopyToOutputDirectory>
40 40   </Content>
41 41   <Content Include="..\..\..\..\ConfigFiles\log4net\DeviceGateway.log4net.xml.template">
42 42     <Link>ConfigFiles\log4net\DeviceGateway.log4net.xml.template</Link>
43 43     <CopyToOutputDirectory>Always</CopyToOutputDirectory>
44 44   </Content>
45 45   <Content Include="..\..\..\..\ConfigFiles\log4net\DeviceGateway.log4net.xml">
46 46     <Link>ConfigFiles\log4net\DeviceGateway.log4net.xml</Link>
47 47     <CopyToOutputDirectory>Always</CopyToOutputDirectory>
48 48   </Content>
49 49   <Content Include="..\..\..\..\ConfigFiles\log4net\DeviceGateway.log4net.xml">
50 50     <Link>ConfigFiles\log4net\DeviceGateway.log4net.xml</Link>
51 51     <CopyToOutputDirectory>Always</CopyToOutputDirectory>
52 52   </Content>
```

Migrate Release Mechanism

- Usually this would not be a topic for the .NET meetup, but...
- it allowed us to bring changes and migration to our customers
- The changes relevant to them were brought in and could just be merged in

Update to v8.3.0.117960...

897898b4 Release Transformation committed Aug 30, 2024 Release/MFC/v8.3

Files Details

Parent 1 → This commit

Filter

Program.cs -2 +34
/Customer/MFC/Source/DeviceGateway.Server/Program.cs

Release

Builds

ConfigFiles

Customer

Client/Source/MFC.Client

Cockpit/Source

ConfigurationTool/Source

MFC

Source

Alarms.Server

DeviceGateway.Domain

DeviceGateway.Server

App.config

CustomDeviceGatewayApplicationLoade...

Customer.DeviceGateway.Server.csproj

Program.cs

TGW.Systems.EnvironmentConfiguration...

MFC.ClientConfiguration.Server

CustomClientConfigurationServerApplic...

Customer.MFC.ClientConfiguration.Serv...

Program.cs

```
1 + using TGW.MFC.DeviceGateway.Server;
2 + using TGW.Systems;
3 + using TGW.Systems.ASPNetCore.IOC;
4 + using TGW.Systems.DataProvider.Odp;
5 + using TGW.Systems.IOC;
6 + using TGW.Systems.Localization;
7 + using TGW.Systems.OData;
8 + using TGW.Systems.Runtime;
9 + using TGW.Systems.Unity;
10 +
11 + using ConfigurationSettings = TGW.Systems.Configuration.ConfigurationSettings;
12 +
13 + #if NET48
14 + using TGW.Rfc1006;
15 + using TGW.MFC.DeviceGateway.MalfunctionMonitor;
16 + using TGW.Systems.DiagnosticsUI;
17 + #endif
18 +
19 + namespace TGW.Customer.DeviceGateway.Server
20 + {
21 +     class Program
22 +     {
23 +         /// <summary>
24 +         /// Main entry-point for this application.
25 +         /// </summary>
26 +         static async Task Main()
27 +         {
28 +             var runtime = RuntimeSelection.Automatic;
29 +             await runtime.RunAsync<CustomDeviceGatewayApplicationLoader>();
30 +             RuntimeServices.Initialize();
31 +             AddIOCAApplicationDescription<Program>(ConfigurationSettings.Applic...
32 +             AddIOCRuntime();
33 +             AddIOCUntityContainer();
34 +             AddIOCServerDefaults();
35 +             AddIOCSysmonMonitor(prov => new ResourceManagerTranslator(Message...
36 +             AddIOCMetrics();
37 +             AddIOCOracleProvider();
38 +             AddIOCCASPNetCore();
39 +             AddIOCCASPNetCore_OData();
40 +             AddIOCCASPNetCore_ODataService<DeviceGatewayODataServiceDefinition...
41 +
42 +             #if NET48
43 +             AddIOCRfc1006();
44 +             AddIOCMalfunctionMonitor();
45 +             #endif
46 +             .RunAsync<CustomDeviceGatewayApplicationLoader>();
47 +         }
48 +     }
49 + }
50 +
```

Completed

17563

Klaus Linzner proposes to merge [User/LZK/master-netstandard](#) into [master](#) All comments resolved

Overview Files Updates Commits Conflicts

All Changes ▾

Filter

2399 changed files



>  Integration.FPRef1

> _Integration.TIS

> Builds

>  Customer

>  gitops_source/openshift

>  Products

> sas-config

>  SharedLibs

Build.Core.Transformation.commit (+)

Build.Core.Transformation.traceback +

IntegrationTest.set

MFC.core.sln +

MFC.runsettings

 MFC.sln

 Nuget.config [rename](#)

Customer.MFC.Client.csproj +1

/_Integration.FPRef1/Client/Source/M

```
97      <ProjectReference Incl
98      <ProjectReference Incl
99      <ProjectReference Incl
```

```
100      <ProjectReference Incl
101      <ProjectReference Incl
102      <ProjectReference Incl
```

MfcClientBootstrapper.cs -6+2

/_Integration.FPRef1/Client/Source/M

```
3 using Microsoft.Practices.Prism;
4 using System;
5 using System.Configuration;
```

```
6 using TGW.Customer.MFC.Client.Res
7 using TGW.Customer.Telegrams;
8 using TGW.MessageGateway.ServiceAge
```

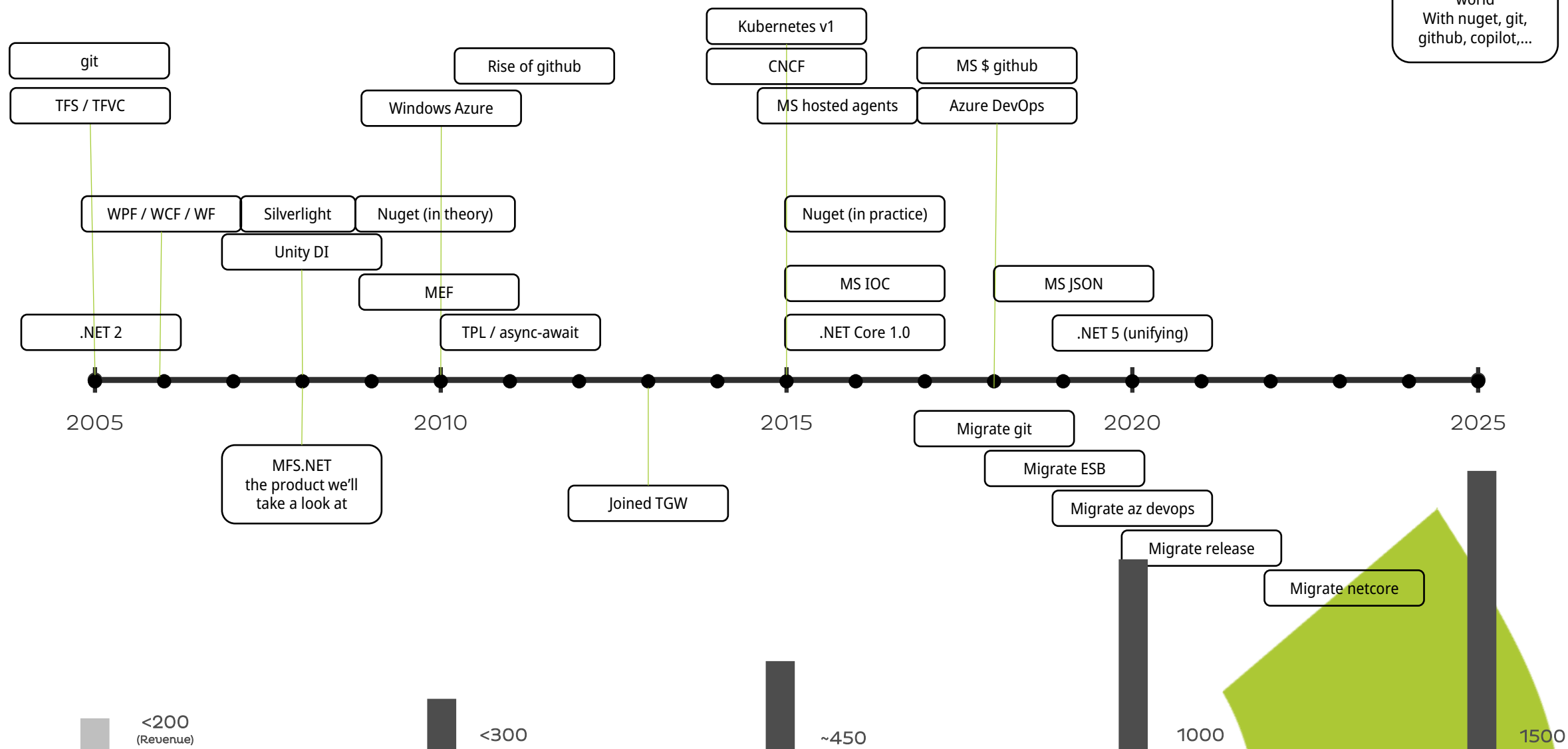
```
28 using TGW.MFC.Windows.Module.Transport;
29 using TGW.MFC.Windows.UI.Ribbon;
30 using TGW.Systems.Configuration;
```

THE ACTUAL .NET CORE MIGRATION

.NET centered timeline



Our shiny .NET 10 world
With nuget, git,
github, copilot,...



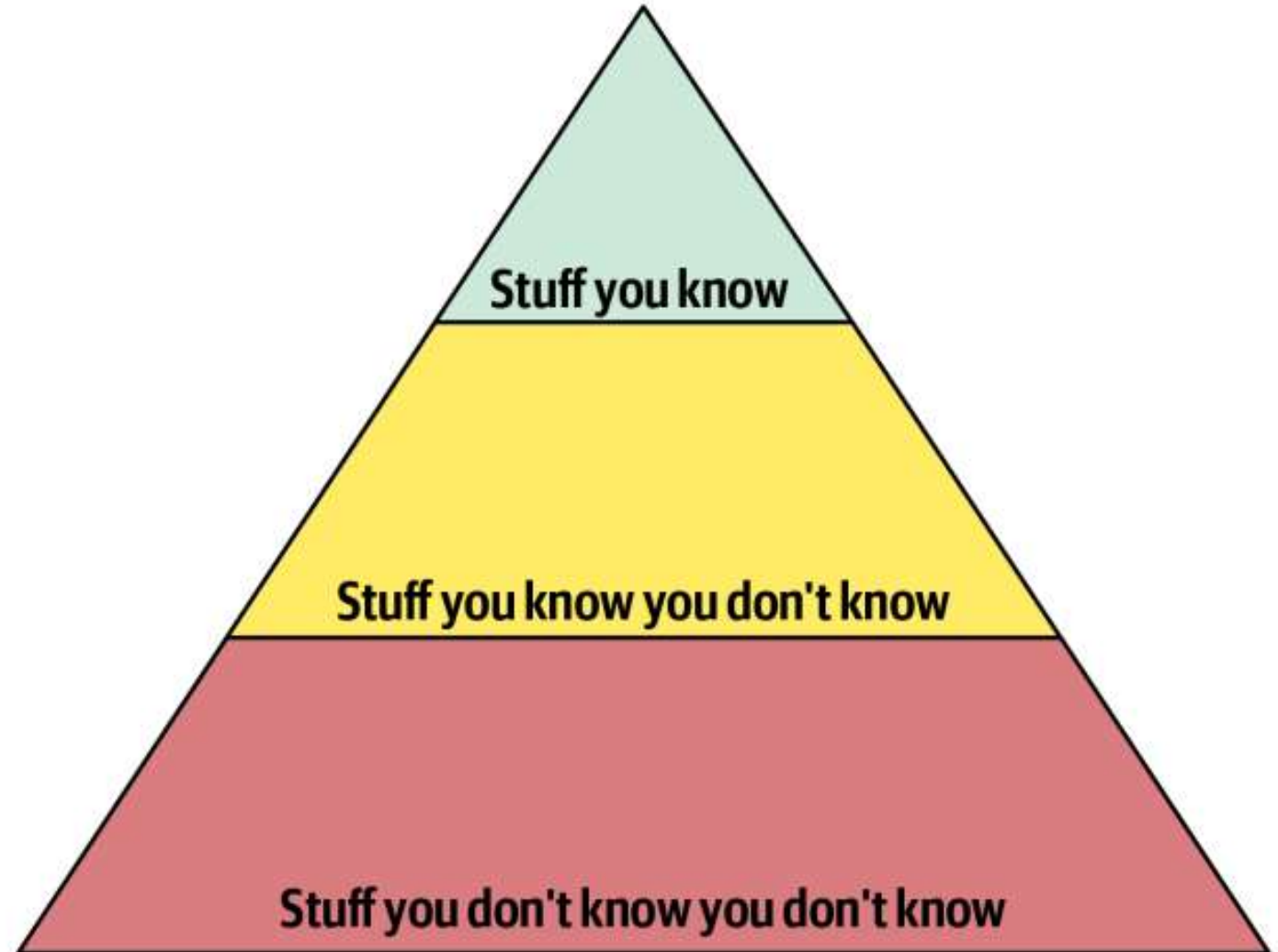
The actual .NET Core migration

Up front

- We did try to do our due diligence on what's ahead of us
- To be as prepared as possible

In the end though

- We had to get there.
Staying on full framework was like riding a dead horse
- If it takes a month or a year, we had to get away from windows onto containers



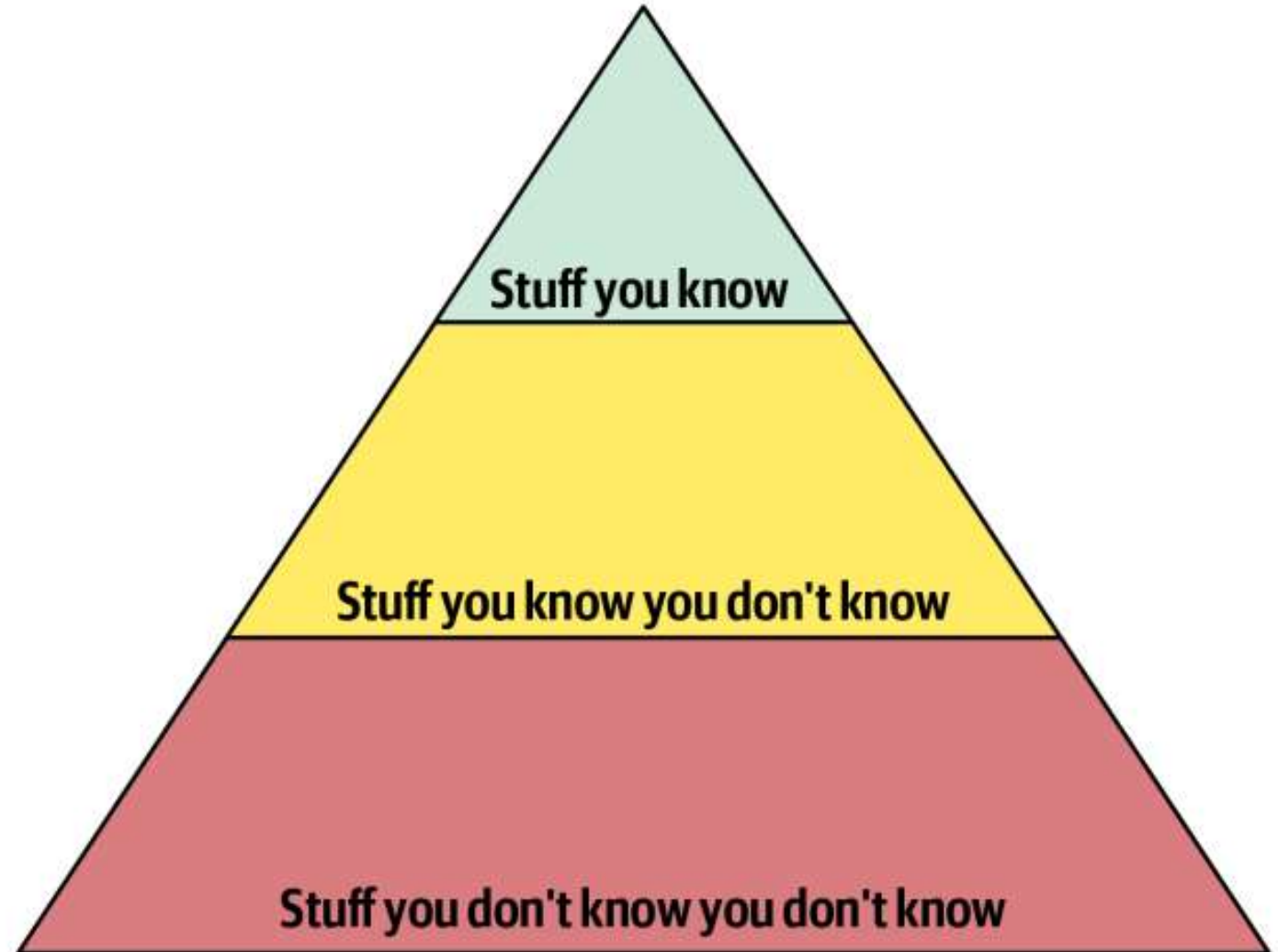
The actual .NET Core migration

■ Hard limit

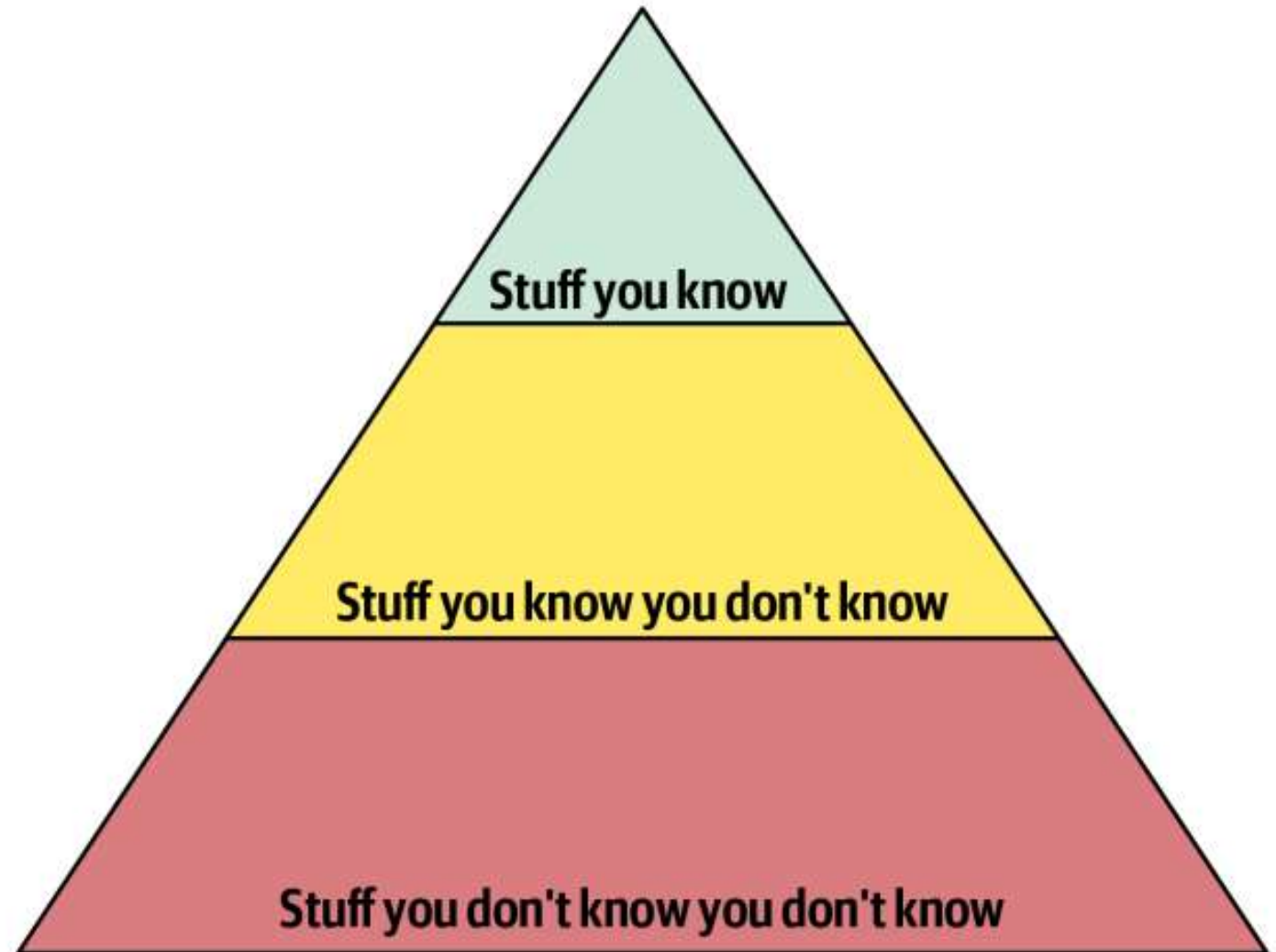
- We had to keep shipping to projects

■ Soft limits

- As little breaking changes as possible (or hide them by shipping the fixes in the customer solution)
- Avoid a new major version as long as reasonably possible
 - A new major version == a new version that needs to be supported by us for ~ 10 years



Stuff you know



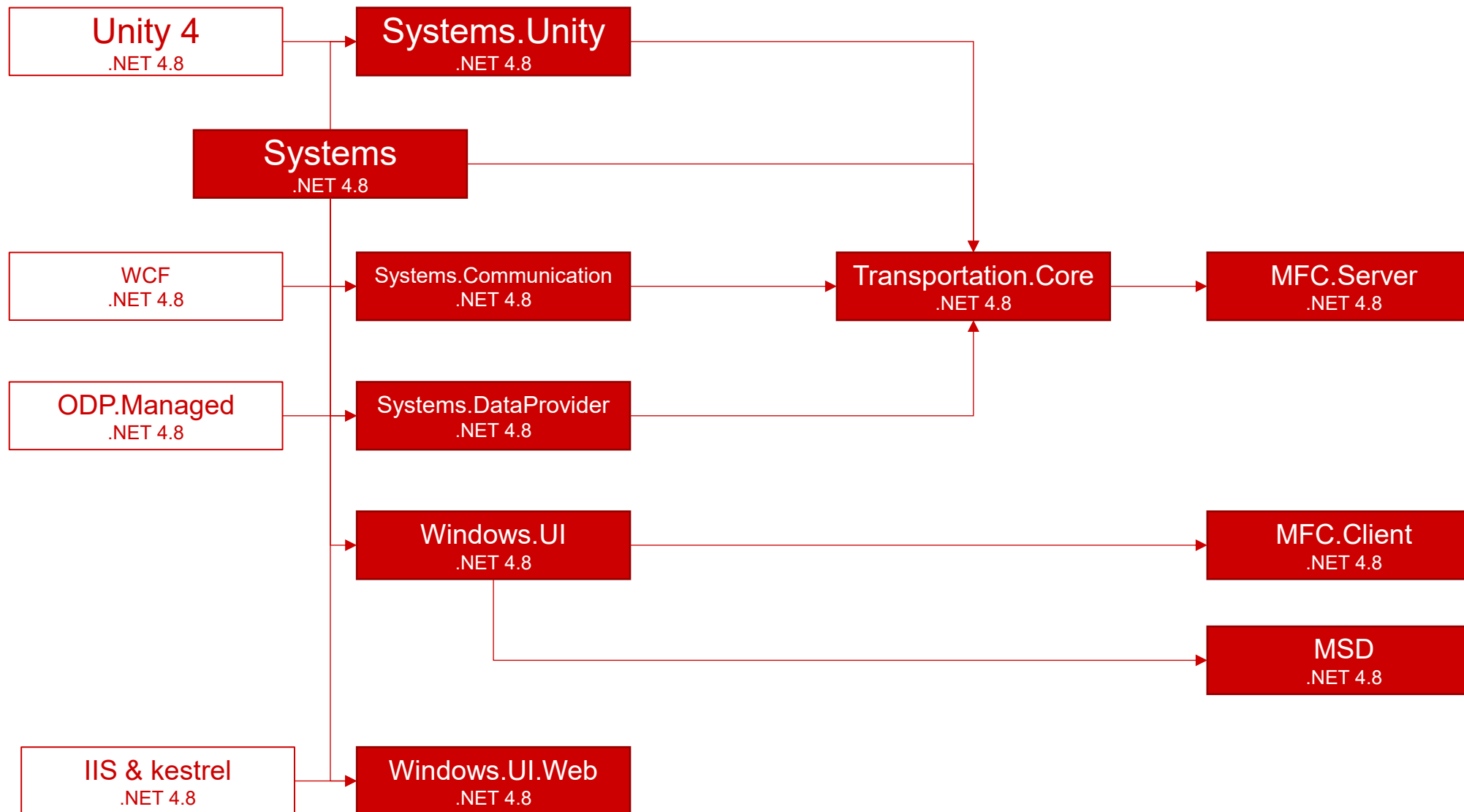
Stuff you know

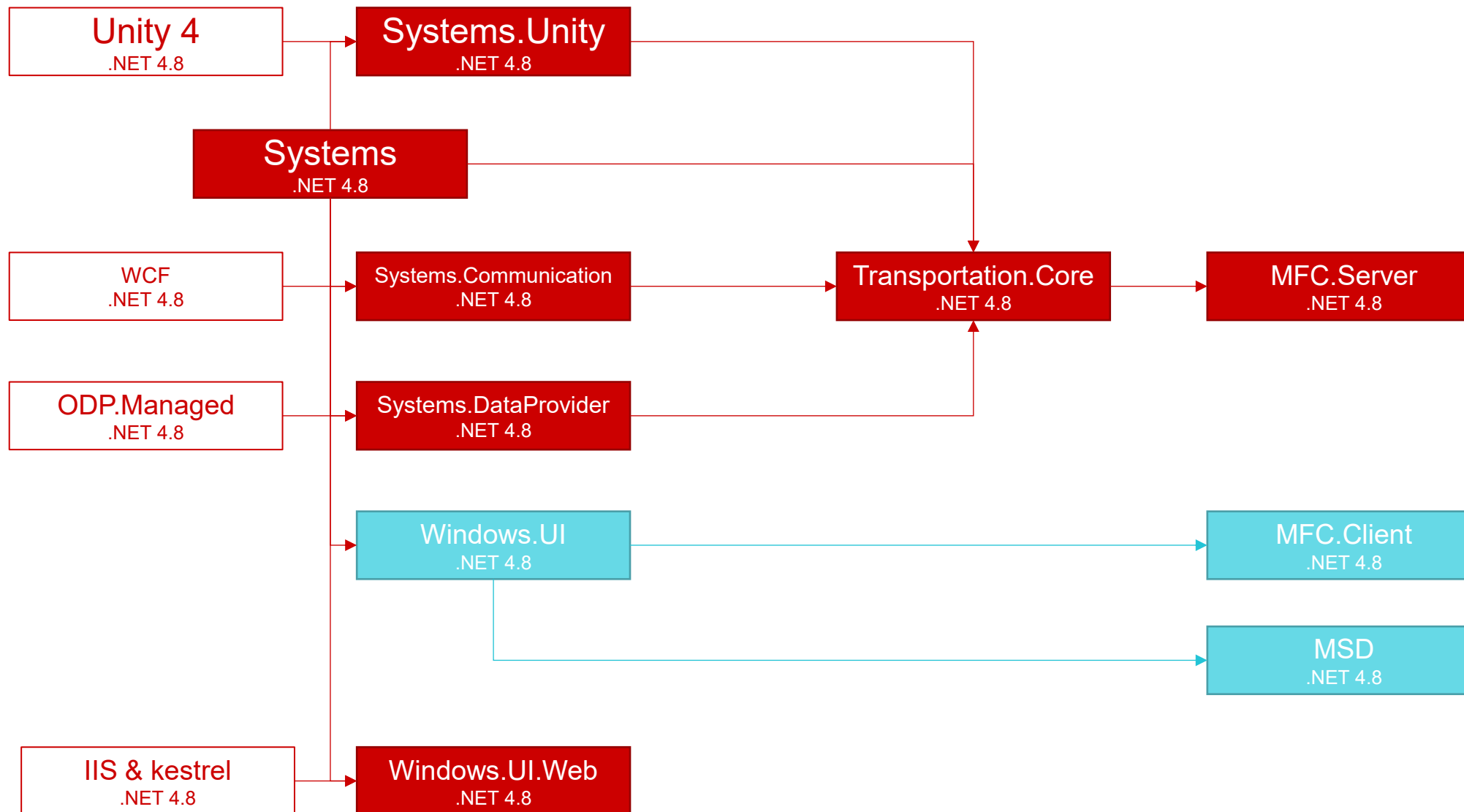
Dependencies

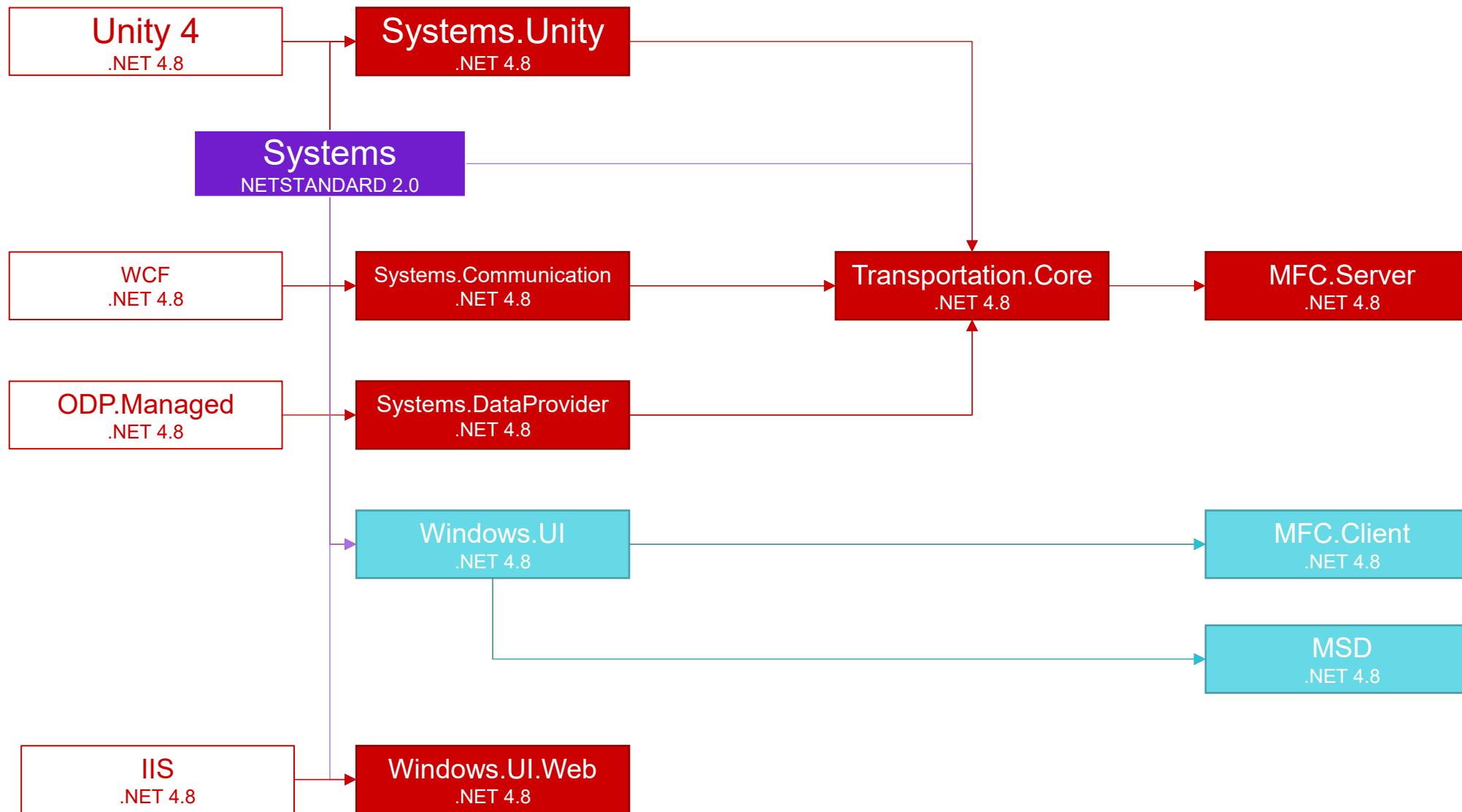
Are there any FullFramework/windows libraries that we use?

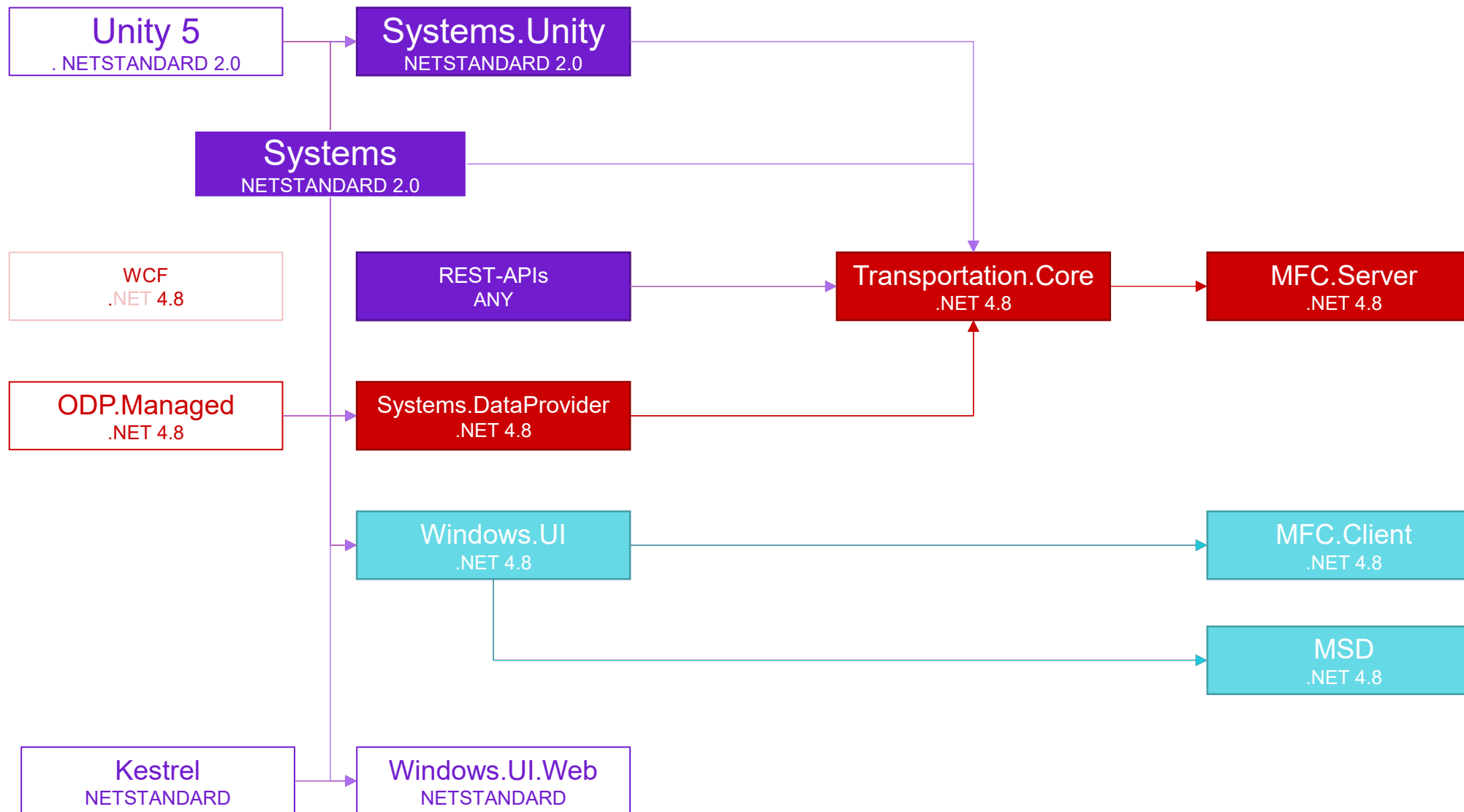
- ODP (Oracle Data Provider for .NET)
- Unity (DI)
- WindowsService Handling
- WPF
- WCF
- ...
- .NET 4.8 BCL

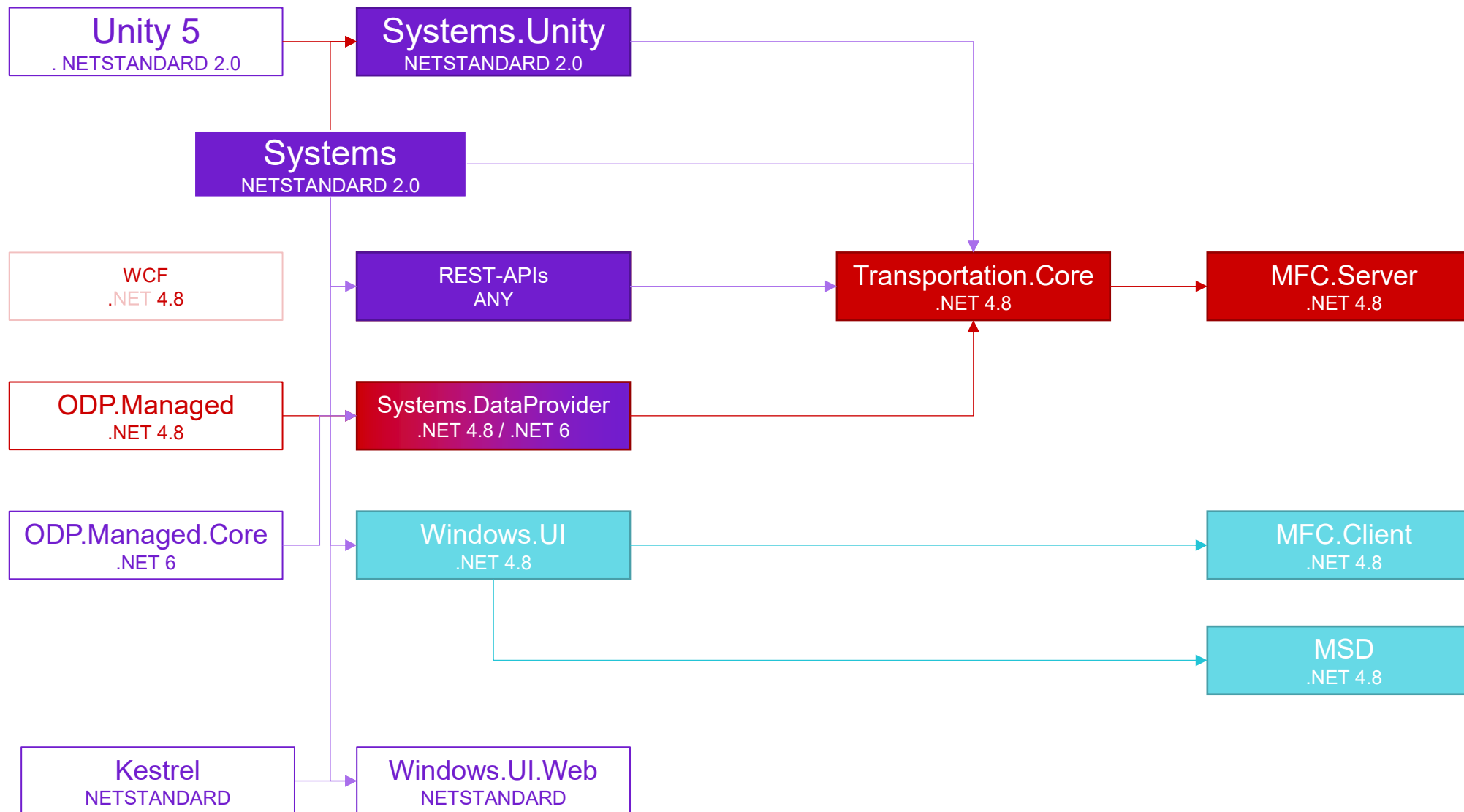






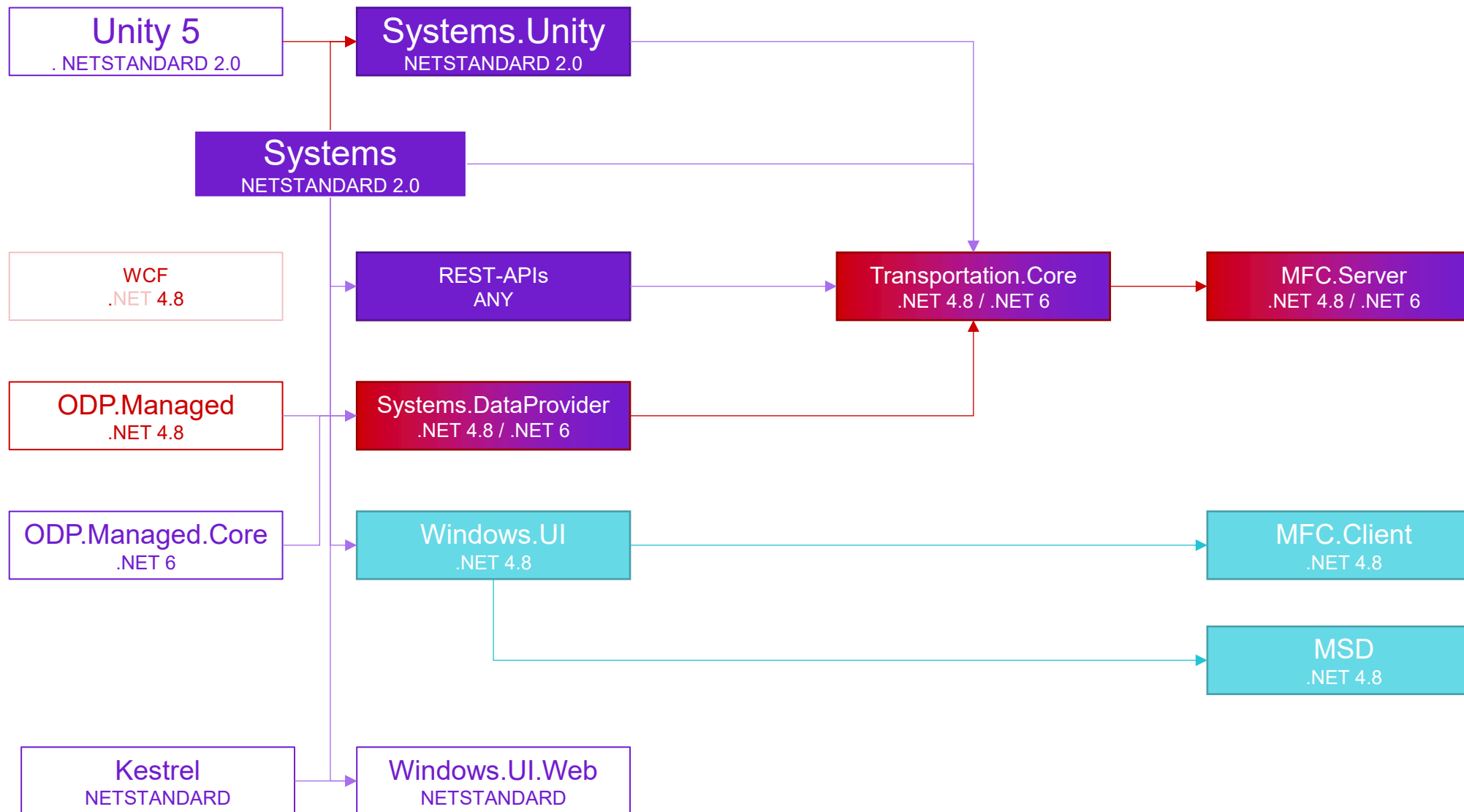






```
<PropertyGroup>
  <TargetFrameworks Condition="'$(BuildTarget)' == ''">net48;net6.0</TargetFrameworks>
  <TargetFramework Condition="'$(BuildTarget)' == 'net48'">net48</TargetFramework>
  <TargetFramework Condition="'$(BuildTarget)' == 'net6.0'">net6.0</TargetFramework>
  <AssemblyName>TGW.Systems.DataProvider.Odp</AssemblyName>
  <RootNamespace>TGW.Systems.DataProvider.Odp</RootNamespace>
</PropertyGroup>
```

```
<ItemGroup Condition="'$(TargetFramework)' == 'net48'">
  <PackageReference Include="Oracle.ManagedDataAccess" Version="19.18.0" />
</ItemGroup>
<ItemGroup Condition="'$(TargetFramework)' == 'net6.0'">
  <PackageReference Include="Oracle.ManagedDataAccess.Core" Version="23.4.0" />
</ItemGroup>
```



Stuff you know

Dependencies

To keep shipping to projects without the need to bring in a new (major) version decision was roughly the following:

- Some tooling / supporting services already had their EOL / sunsetting on the roadmaps – they will remain as-is if there are any hoops we need to jump through
- Windows Client is used only as internal commissioning tool by realization teams -> we leave it as is – can be started from IDE
- Move to netstandard 2.0 as much as possible
- Dual Compilation will be required for some areas – as little as possible
- Conditional code compilation only if everything else fails



Stuff you know

Dependencies

- Remember the line:

„We don't want to deal with your internal layerings – hide your complexity and provide us functionality so we only have to reference one assembly“

- We did know: The top level „Systems“ project needs a lot of work to refactor.
 - This was definitely underestimated

```
Include="Configuration\SharedSe
er</SubType>
e>

Include="App.Metrics" Version="
Include="Castle.Core" Version="
Include="log4net" Version="2.0.
Include="Microsoft.AspNetCore.M
Include="Microsoft.AspNetCore.F
Include="Microsoft.AspNetCore.S
Include="Microsoft.Bcl.AsyncInt
Include="Microsoft.CodeAnalysis
Include="Microsoft.Extensions.F
Include="MongoDB.Driver" Versio
Include="Newtonsoft.Json" Versi
Include="StackExchange.Redis" V
Include="System Buffers" Versio

Include="System.Configuration.C
Include="System.IO.Pipelines" V

Include="System.Memory" Version
Include="System.Runtime.Interop
Include="System.Threading.Tasks
Include="Unity" Version="5.11.1

build process, add your task in
extension points exist, see Micro
eBuild">

Build">

e Include="System.Runtime.Compil
W Systems.Diagnostics\li</ Paramete
e>

44 <<ItemGroup>
45 <<EmbeddedResource Include="Configuration\SharedSet
46 <<SubType>Designer</SubType>
47 <</EmbeddedResource>
48 <</ItemGroup>
49 <<ItemGroup>

50 <<PackageReference Include="log4net" Version="2.0.1
51 + <<PackageReference Include="Microsoft.Extensions.Co
52 + <<PackageReference Include="Microsoft.Extensions.Co
53 + <<PackageReference Include="Microsoft.Extensions.De
54 + <<PackageReference Include="Microsoft.Extensions.Lo

55 <<PackageReference Include="Newtonsoft.Json" Versio
56 <<PackageReference Include="System.Buffers" Version
57 + <<PackageReference Include="System.ComponentModel.A
58 <<PackageReference Include="System.Configuration.Co
59 + <<PackageReference Include="System.Diagnostics.Perf
60 <<PackageReference Include="System.IO.Pipelines" Ve
61 + <<PackageReference Include="System.Linq" Version="4
62 <<PackageReference Include="System.Memory" Version=
63 + <<PackageReference Include="System.Text.Encoding.Co
64 <<PackageReference Include="System.Threading.Tasks.

65 <</ItemGroup>
66 <<!-- To modify your build process, add your task ins
67 <<!-- Other similar extension points exist, see Micro
68 <<Target Name="BeforeBuild">
69 <</Target>
70 <<Target Name="AfterBuild">
71 <</Target>
72 <<-->
73 <<ItemGroup>
74 <<AssemblyAttribute Include="System.Runtime.Com
75 <<Parameter1>TGW Systems.Diagnostics\li</ Paramete
```

Stuff you know path separators & casing

That's kind of an easy one:

Actual production code was in
general fine with proper usage,

Some adjustments in test cases
where shortcuts were taken

Some adjustments on production
code because of case-sensitive
filesystem

```
public async Task ServiceCanHandleRequest()
{
    var mockAgent = new MockStreamsConnection("Service");
    mockAgent.MessageDispatcher = new RabbitMessageDispatcher(mockAgent);
    mockAgent.ProxyFactory = new RabbitStreamsProxyFactory();
    var service = new MockIwacsTestService();

    var facade = mockAgent.CreateServiceFacade<IIwacsTestServiceDefinition>(service);

    var message = ReadFromFile("Proxy\\IWACS\\StartOperationRequest.json", "StartOperationRequest");
    var message = ReadFromFile(Path.Combine("Proxy", "IWACS", "StartOperationRequest.json"), "StartOperationRequest");

    Assert.IsNull(service.LastReceived);

    await mockAgent.ReceiveMessage(message);

    Assert.IsNotNull(service.LastReceived);
    Assert.AreEqual("525642c0-8715-4a5c-b371-259f5738f5ae", service.LastReceived.CorrelationId);
    Assert.AreEqual("One", service.LastReceived.Payload.PropertyOne);
    Assert.AreEqual(2, service.LastReceived.Payload.PropertyTwo);
    Assert.IsTrue(DateTime.Parse("2020-09-08T23:45:59.8157065+02:00").ApproximatelyEquals(service.LastReceived.Payload.Timestamp));
}

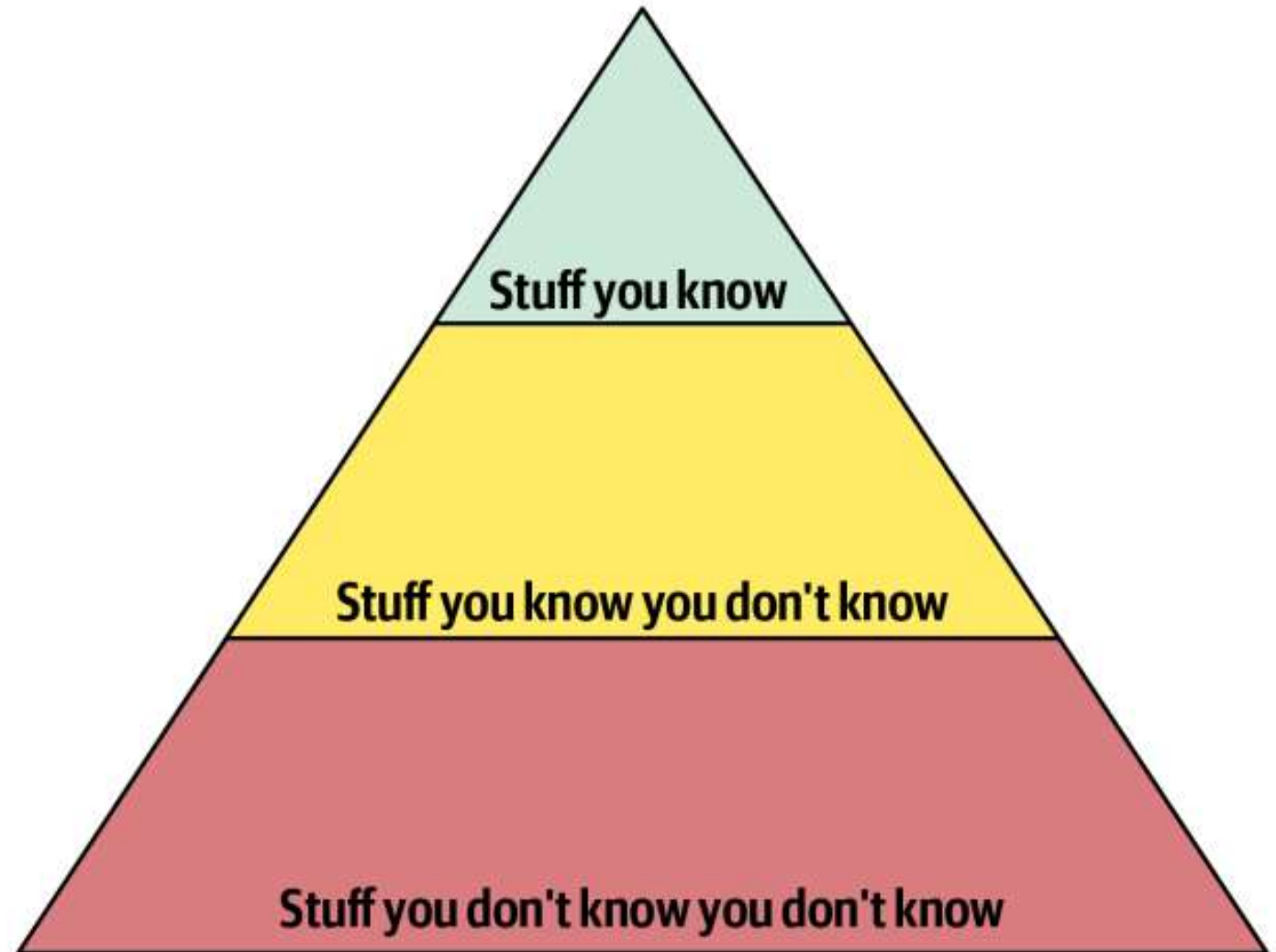
private Message<JToken> ReadFromFile(string file, TopicId topic, PartyId sender)
{
    var fileToRead = file;
    var fileToRead = IoOperations.AdjustPathSeparator(file);
    var content = File.ReadAllText(fileToRead);

    var message = JsonSerializer.DeserializeObject<Message<JToken>>(content);
    message.Topic = topic;
    message.Sender = sender;

    return message;
}

[TestMethod]
```

Stuff you know
you don't know



Stuff you know you don't know



Windows vs Containerized Linux

What's actually different if a large application runs in container on linux instead of windows?

Expected

- Different debugging
- Different performance analysis
- Garbage Collection
- ThreadPool
- Application Shutdown in containerized environment
- Some quirks we don't know

Stuff you know you don't know

Windows vs Containerized Linux

From performance point of view: It went much smoother than expected

- Guessed that GC will bite us -> nope
- Guessed that ThreadPool will bite us -> nope
- Guessed that Application Shutdown will bite us -> nope

The „Stuff you know you don't know“ was a pleasant surprise

- IO in general went smooth
- Runtime handled most of the underlying differences great



Stuff you know you don't know Locks

Some hot areas with lock statements.

If there's no contention linux works at least as well as windows.

If there's heavy contention in hot paths performance breaks down a lot compared to windows.

<https://github.com/dotnet/runtime/issues/8255>

<https://github.com/dotnet/runtime/issues/8931>

From application point of view there were no issues.

Did not need to dive deep to diagnose: took one process dump and opened it (sorry that I don't have a screenshot of it anymore)

The screenshot shows a GitHub issue titled "Locks slow on Linux with lots of threads #8255" which is marked as "Closed". The issue was created by user "kouvel" on June 28, 2017. The discussion includes a detailed analysis of lock contention on Linux, comparing it to Windows, and proposing a solution to allow spinners to preempt waiters. A table of "Default spin heuristics" is also visible, showing performance metrics for various thread counts on both Windows and Linux.

Issue Title: Locks slow on Linux with lots of threads #8255 (Closed)

Contributor: kouvel (on Jun 28, 2017)

Text: I have dug a little deeper and I believe I have found the root cause. Some facts:

- In the example given, with a design where there is no spinning and when there is a waiter, a waiter must acquire the lock, there is a deterministic lock convoy. Every release of the lock requires that a waiter acquire it, and the context switching kills the perf.
- This situation is occurring on Windows and Linux.
- However, Windows' priority boost for released threads allows a waiter to get a lock sooner, and release another waiter sooner, etc., eventually removing all waiters. So on Windows, the lock convoy is fixing itself due to scheduling heuristics in Windows. Given enough waiters, theoretically, this would be an issue on Windows as well.
- On Linux, once the lock convoy begins, there is no end to it.

Text: As such, I believe the only solution to this is the first one I suggest:

- Allow spinners to preempt waiters and acquire the lock. This problem of starving waiters

Text: Some data:

- Reducing spinning actually makes the issue worse, because frequency with which threads become waiters, which initiate
- Adding SwitchToThread/Sleep(0) helps only to the extent that becoming a waiter (but it should be done anyway)

Text: @jkotas, @stephentoub, @vancem, what are your thoughts on else I should test to see if I'm missing something?

Table: Default spin heuristics

Threads	Windows locks/ms	Linux locks/ms
1	40562.15	40479.00
1	50812.96	52917.10
1	49464.75	48424.61
2	49665.72	47385.07
2	49602.25	46933.33
2	49617.80	46855.11
3	48307.62	46370.60
3	46291.77	44576.09
3	48292.34	44423.89
6	44089.59	34304.88
6	44827.27	34329.09
6	44065.69	34516.30
12	34774.26	20239.20
12	36423.95	29188.46
12	35451.14	23091.97
24	35274.48	27140.28
24	36145.63	25053.76
24	34648.29	22950.01
48	25808.94	0.67
48	36862.25	0.72
48	30344.51	0.85
96	25332.65	4585.24
96	38030.90	0.57
96	76290.54	0.49
192	25253.54	0.67
192	42564.09	0.85
192	48377.46	0.65

Stuff you know you don't know Lock

The hottest path: log4net

Under load applications writes 1500+ messages / second from 200+ threads

Because of this we had to get rid of most of/ all the default log4net appenders/formatters in the past and write our own appender that do their work outside central lock paths - buffer, aggregate, format specifically

This lock was at log4nets core – so got rid of log4net entirely (at least when running containerized. Left it in for backward compatibility on net48)

```
public static IServiceCollection AddIOCLogging(this IServiceCollection collection)
{
    if (collection == null) throw new ArgumentNullException(nameof(collection));

    return RuntimeDescription.IsContainer
        ? collection.AddIOCConsoleLogging()
        : collection.AddIOCLoggingLog4Net();
}
```

Stuff you know you don't know Lock

In similar direction: StringComparison behave differently with vastly different runtime diagnostics:

Default .NET operations take culture settings into account.

Windows on 4.8: NLS

Linux/netcore: ICU

ICU in linux is

- more correct,
- more unicode compliant,
- more consistent,
- more complex
- takes much more time

ement.cs -8+8

/Source/Core/TransportElements/ConveyorTransportElement.cs

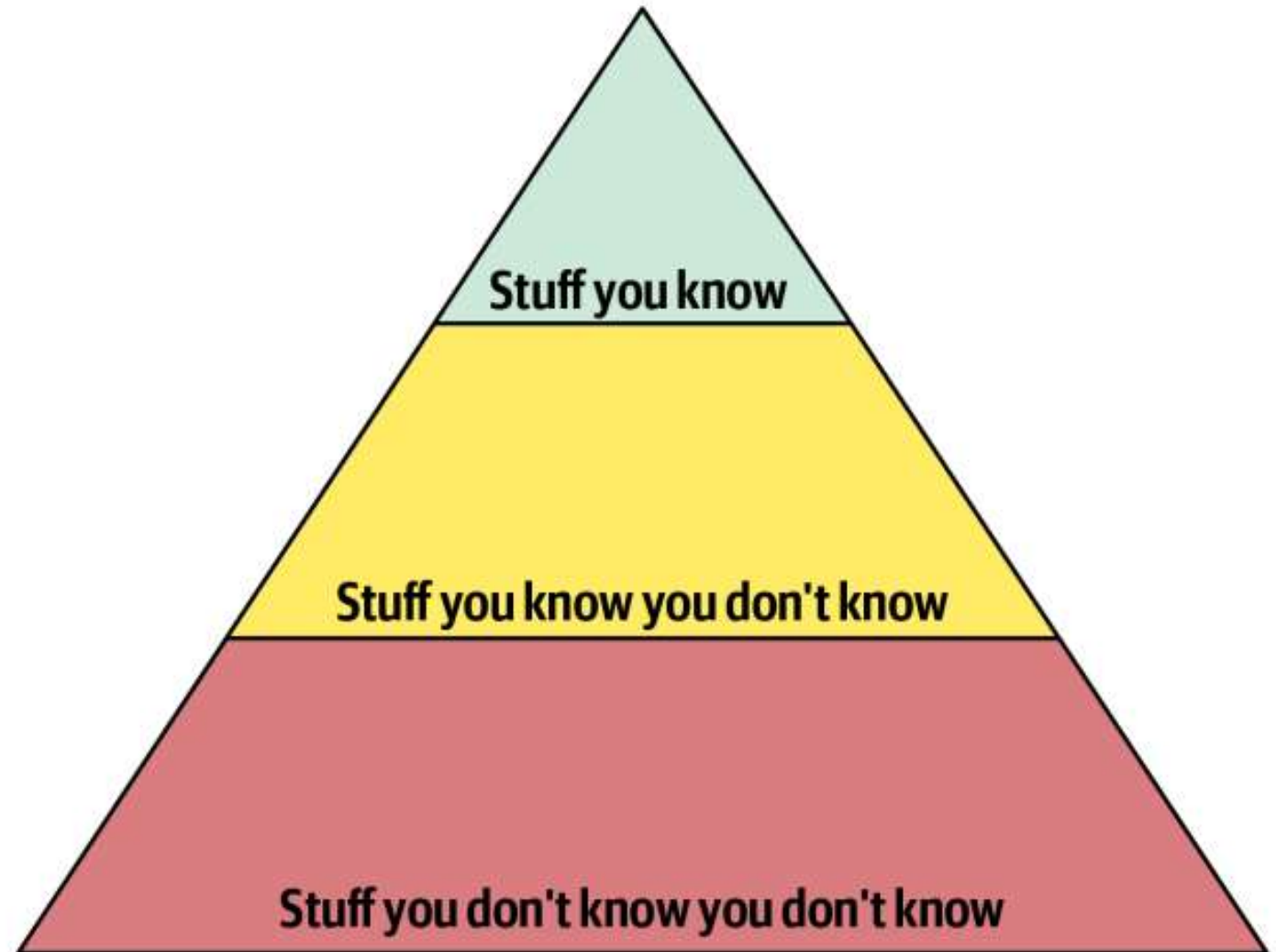
```
foreach (string sequencePoint in tschaliService.ConfiguredSequencePoints.Keys)
{
    if (sequencePoint.StartsWith(FullName))
    if (sequencePoint.StartsWith(FullName, StringComparison.Ordinal))
    {
        sequencePointsToCascades[sequencePoint] = cascade;
    }
}
```

```
foreach (Doorway doorway in cascade.Doorways)
{
    var electionAddress = doorway.ElectionAddress ?? doorway.Name;
    if (electionAddress.StartsWith(FullName))
    if (electionAddress.StartsWith(FullName, StringComparison.Ordinal))
    {
        electionPoints[electionAddress] = doorway;
    }
    if (doorway.Name.StartsWith(FullName))
    if (doorway.Name.StartsWith(FullName, StringComparison.Ordinal))
    {
        doorways[doorway.Name] = doorway;
    }
}
```

```
}
```

```
//restore bundle allocations
foreach (string allocationValue in staticInfos.Where(kvp => kvp.Key.StartsWith(RECOVERY_
foreach (string allocationValue in staticInfos.Where(kvp => kvp.Key.StartsWith(RECOVERY_
{
    using (XmlReader reader = XmlReader.Create(new StringReader(allocationValue),
        new XmlReaderSettings
```

Stuff you don't know
you don't know



Stuff you don't know you don't know

■ Something really small but nice:

Datetime resolution improved dramatically so we don't need to fall back to explicit underlying OS calls for detailed timestamps in logs

```
public static class PreciseDateTime
{
    /// <summary>
    /// Returns the current DateTime with sub-ms accuracy.
    /// </summary>
    public static DateTime Now
    {
        get
        {
            if (RuntimeDescription.IsNetCore)
            {
                return DateTime.Now;
            }

            GetSystemTimePreciseAsFileTime(out long fileTime);
            return DateTime.FromFileTime(fileTime);
        }
    }

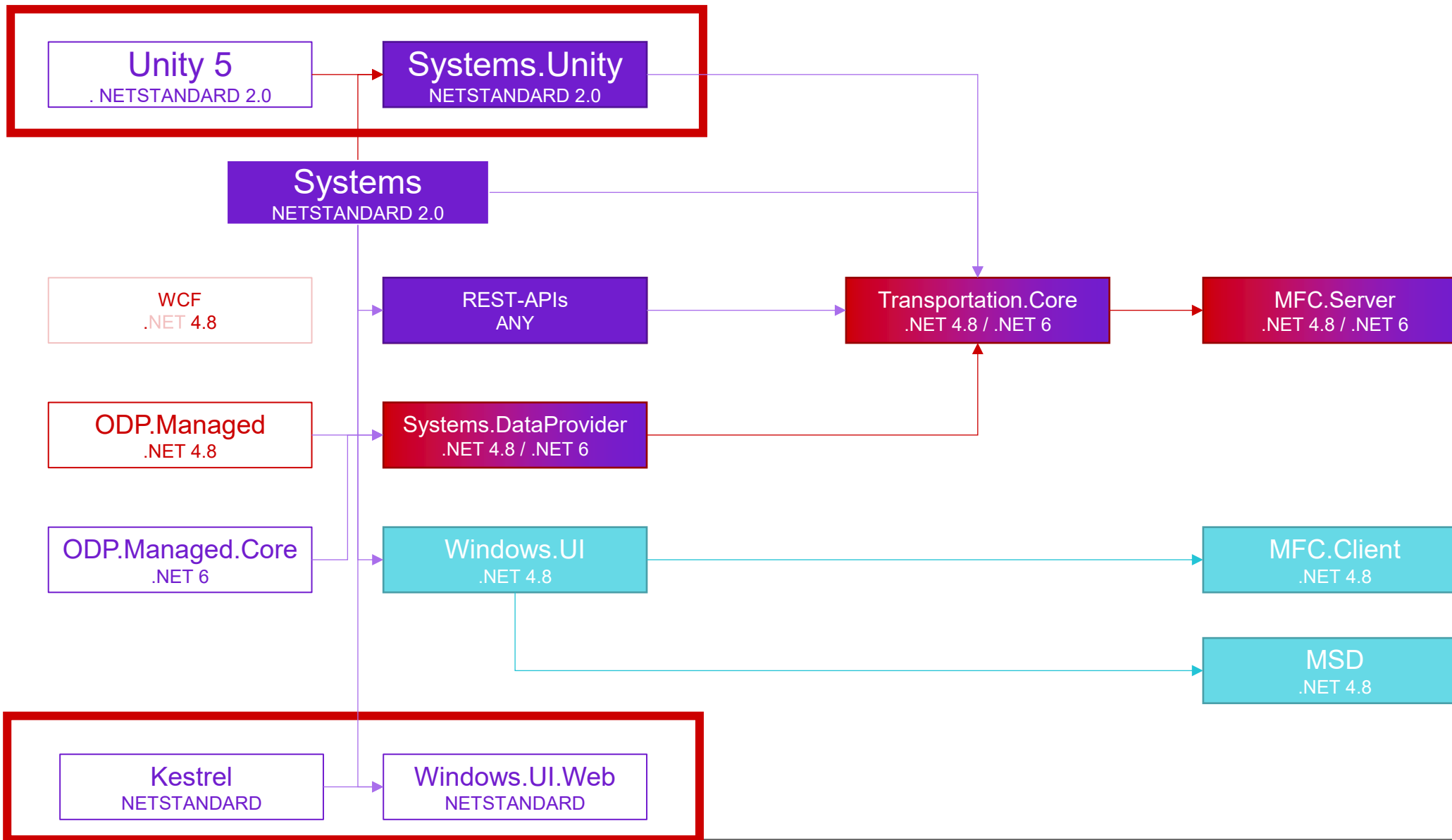
    [DllImport("Kernel32.dll", CallingConvention = CallingConvention.Winapi)]
    static extern void GetSystemTimePreciseAsFileTime(out long filetime);
}
```

Stuff you don't know you don't know

Minor issue (that should've been known
up front - nevertheless)

Environment.NewLine

```
-----  
36 36  
37 37     private static readonly string keepAliveTargetBlockName = "Sequencing";  
38 38  
39 39 +     private static readonly string CRLF = "\r\n";  
40 40 +  
39 41     private static TcpSourceAdapterOptions<string> GetTcpAdapterOptions(TcpClientConfiguration definition)  
40 42     {  
41 43         string adapterName = "TcpDuplex";  
-----  
43 45         options.Host = definition.RemoteHost;  
44 46         options.Port = definition.RemotePort;  
45 47  
46 48 -     options.Tokenizer = new DelimitedStringTokenizer(Environment.NewLine);  
47 49 +     options.Tokenizer = new DelimitedStringTokenizer(CRLF);  
48 50         options.BufferSize = 128;  
49 51
```



Stuff you don't know you don't know

ASPNET.Core in netstandard

- UP FRONT: I hope I still have all the details correctly in mind
- AspNetCore.Mvc is available as netstandard 2 package
 - Assumption: It can be used in net6 as well as in net48
 - It compiles.
 - Some stuff works.

 Microsoft.AspNetCore.Mvc.Abstractions ✓

Version 2.3.9

> Info ASP.NET Core MVC abstractions and interfaces for action invocation and dispatching

✓ Frameworks and Dependencies

- .NETStandard 2.0
 - Microsoft.AspNetCore.Routing.Abstractions ≥ 2.3.0
 - Microsoft.Net.Http.Headers ≥ 2.3.8

Stuff you don't know you don't know

ASP.NET.Core in netstandard

- UP FRONT: I hope I still have all the details correctly in mind
- AspNetCore.Mvc is available as netstandard 2 package
 - Assumption: It can be used in net6 as well as in net48
 - It compiles.
 - Some stuff works.
 - It's not supposed to work
 - MissingMethodException et al.

<https://learn.microsoft.com/en-us/aspnet/core/fundamentals/target-aspnetcore?view=aspnetcore-10.0&tabs=visual-studio#use-the-aspnet-core-shared-framework>

Use the ASP.NET Core shared framework

With the release of .NET Core 3.0, many ASP.NET Core assemblies are no longer published to NuGet as packages. Instead, the assemblies are included in the `Microsoft.AspNetCore.App` shared framework, which is installed with the .NET SDK and runtime installers. For a list of packages no longer being published, see [Remove obsolete package references](#).

As of .NET Core 3.0, projects using the `Microsoft.NET.Sdk.Web` MSBuild SDK implicitly reference the shared framework. Projects using the `Microsoft.NET.Sdk` or `Microsoft.NET.Sdk.Razor` SDK must reference ASP.NET Core to use ASP.NET Core APIs in the shared framework.

To reference ASP.NET Core, add the following `<FrameworkReference>` element to your project file:

```
XML Copy
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <TargetFramework>netcoreapp3.1</TargetFramework>
  </PropertyGroup>

  <ItemGroup>
    <FrameworkReference Include="Microsoft.AspNetCore.App" />
  </ItemGroup>

</Project>
```

Stuff you don't know you don't know ASPNET.Core in netstandard

- Lots and lots of desperate attempts to make it work
- This led to dual compilation again
 - At that's not the worst part

```
<ItemGroup Condition=" '$(TargetFramework)' == 'net48'">
  <PackageReference Include="Microsoft.AspNetCore.Server.Kestrel" Version="2.2.0" />
  <PackageReference Include="Microsoft.AspNetCore.Hosting" Version="2.2.0" />
  <PackageReference Include="Microsoft.AspNetCore.Mvc" Version="2.2.0" />
  <PackageReference Include="Microsoft.Extensions.Configuration" Version="6.0.0" />
  <PackageReference Include="System.ComponentModel.Annotations" Version="5.0.0" />

  <PackageReference Include="Microsoft.AspNetCore.Diagnostics" Version="2.2.0" />
  <PackageReference Include="Microsoft.AspNetCore.Mvc.Razor" Version="2.2.0" />
  <PackageReference Include="Microsoft.AspNetCore.Razor" Version="2.2.0" />
  <PackageReference Include="Microsoft.AspNetCore.StaticFiles" Version="2.2.0" />
  <PackageReference Include="Microsoft.CodeAnalysis.Razor" Version="2.2.0" />
  <PackageReference Include="Microsoft.Extensions.FileProviders.Embedded" Version="3.1.3" />
</ItemGroup>
<ItemGroup Condition=" '$(TargetFramework)' == 'net6.0'">
  <FrameworkReference Include="Microsoft.AspNetCore.App" />
</ItemGroup>
```

Stuff you don't know you don't know ASPNET.Core in netstandard

- ☛ Lots and lots of desperate attempts to make it work
- ☛ This led to dual compilation again
 - ☛ At that's not the worst part
 - ☛ That means essentially targeting aspnetcore 2.x and aspnetcore 6 from a common code base

```
.ConfigureServices(services :IServiceCollection =>
{
    ....services.ImportFrom(applicationServiceCollection);
    ....services.AddIOCTraceRequest();
    #if NET48
    ....services.AddIOCFix18438MiddleWare();
    ....var mvcBuilder = services.AddMvc(options => {
    ....|....options.EnableEndpointRouting = false;
    ....});
    ....mvcBuilder.SetCompatibilityVersion(CompatibilityVersion
    ....RunIIOCASPNetCore_AddRouting48(provider, mvcBuilder);
    #endif

    #if NET6_0_OR_GREATER
    ....services.AddRouting();
    ....var mvcBuilder = services.AddRazorPages();
    ....RunIIOCASPNetCore_AddRouting60(provider, mvcBuilder);
    #endif

    ....RunIOCConfigureServices(provider, services);

}).Configure(configureApp :ApplicationBuilder =>
```

Stuff you don't know you don't know ASPNET.Core in netstandard

- 👉 Lots and lots of desperate attempts to make it work
- 👉 This led to dual compilation again
 - 👉 At that's not the worst part
 - 👉 That means essentially targeting aspnetcore 2.x and aspnetcore 6 from a common code base
 - 👉 And providing an API layer in-between

```
collection.AddIOCasPNetCore_AddRouteRegistration( description: "AddHealthRoute", useEndpointRoute: (appBuilder, routeBuilder) =>
{
    routeBuilder.MapGet( pattern: "health",
        requestDelegate: async context =>
        {
            await RunIOC_HealthChecks(appBuilder.ApplicationServices, context);
        });
});
return collection;
```

```
#if NET6_0_OR_GREATER
using ...

namespace TGM.Systems.ASPNetCore.IOC
{
    [Messages, ListenerClass]
    public static partial class AddIOCasPNetCoreExtensions
    {
        /// In NET4.8 can register a lambda that will be executed during AddRouting ...
        [Messages, ListenerClass]
        public static IServiceCollection AddIOCasPNetCore_AddRouting(this IServiceCollection applicationServiceCollection,
            string description,
            Action<IMvcBuilder> addMvc) { ... }

        /// <summary>
        /// Adds an endpoint route to AspNetcore 6.0 on .NET Core 6 - will be called during startups call to UseEndpoints
        /// </summary>
        [Messages, ListenerClass]
        public static IServiceCollection AddIOCasPNetCore_AddRouteRegistration(this IServiceCollection applicationServiceCollection,
            string description,
            Action<IApplicationBuilder, IEndpointRouteBuilder> useEndpointRoute)
        {
            applicationServiceCollection.AddTransient<IOCasPNetCore_UseRouteNet60Registration>(_ => new IOCasPNetCore_UseRouteNet60Registration(description, useEndpointRoute));
            return applicationServiceCollection;
        }
    }
}
```

```
#if NET48
using ...

namespace TGM.Systems.ASPNetCore.IOC
{
    [Messages, ListenerClass]
    public static partial class AddIOCasPNetCoreExtensions
    {
        /// In NET4.8 can register a lambda that will be executed after AddMvc ...
        [Messages, ListenerClass]
        public static IServiceCollection AddIOCasPNetCore_AddRouting(this IServiceCollection applicationServiceCollection,
            string description,
            Action<IMvcBuilder> addMvc) { ... }

        /// <summary>
        /// Adds a MVC route to AspNetcore 2.x on .NET 48 - will be executed during startups call to UseMvc
        /// </summary>
        [Messages, ListenerClass]
        public static IServiceCollection AddIOCasPNetCore_AddRouteRegistration(this IServiceCollection applicationServiceCollection,
            string description,
            Action<IApplicationBuilder, IRouteBuilder> useMvc)
        {
            applicationServiceCollection.AddTransient<IOCasPNetCore_UseRouting48Registration>(_ => new IOCasPNetCore_UseRouting48Registration(description, useMvc));
            return applicationServiceCollection;
        }
    }
}
```

Stuff you don't know you don't know IOC impact

- Refactoring for central / core library was required to come to netstandard
- Which meant we need IOC
- Unity 4 kind of provided something like it
 - Not quite dead –they brought out a unity 5 with netstandard support

Unity 5.11.10

Prefix Reserved

.NET Core 1.0 .NET Standard 1.0 .NET Framework 4.0

.NET CLI PMC PackageReference CPM Paket CLI Script & Interactive File-based Apps Cake

<PackageReference Include="Unity" Version="5.11.10" />

Copy

For projects that support PackageReference, copy this XML node into the project file to reference the package.

README Frameworks Dependencies Used By Versions Release Notes

Include prerelease Include vulnerable Include deprecated

Version	Downloads	Last Updated
5.11.10	22,350,069	1/29/2021
5.11.9	1,881,409	11/23/2020
5.11.8	75,825	11/23/2020
5.11.7	3,707,013	5/25/2020
5.11.6	1,137,596	4/23/2020
5.11.5	440,793	4/2/2020
5.11.4	1,340,040	2/24/2020
5.11.3	1,898,816	1/6/2020
5.11.2	655,320	12/6/2019
5.11.1	5,043,646	6/20/2019
5.10.3	1,547,506	4/4/2019
5.10.2	299,337	3/23/2019
5.10.1	116,676	3/19/2019
5.10.0	161,202	3/18/2019
5.9.8	80,978	3/17/2019
5.9.7	481,584	3/4/2019
5.9.6	42,095	3/2/2019
5.9.5	271,083	2/18/2019
5.9.4	107,360	2/9/2019
5.9.3	845,542	1/30/2019

Downloads Full stats

Total 112.6M

Current version 22.4M

Per day average 42.4K

About

Last updated 1/29/2021

Project website

License info

Download package (1.25 MB)

Open in NuGet Package Explorer

Open in NuGet Trends

Report package

Owners Contact owners

eniks

unitycontainer

esadovoi

Unity Container

unitycontainer

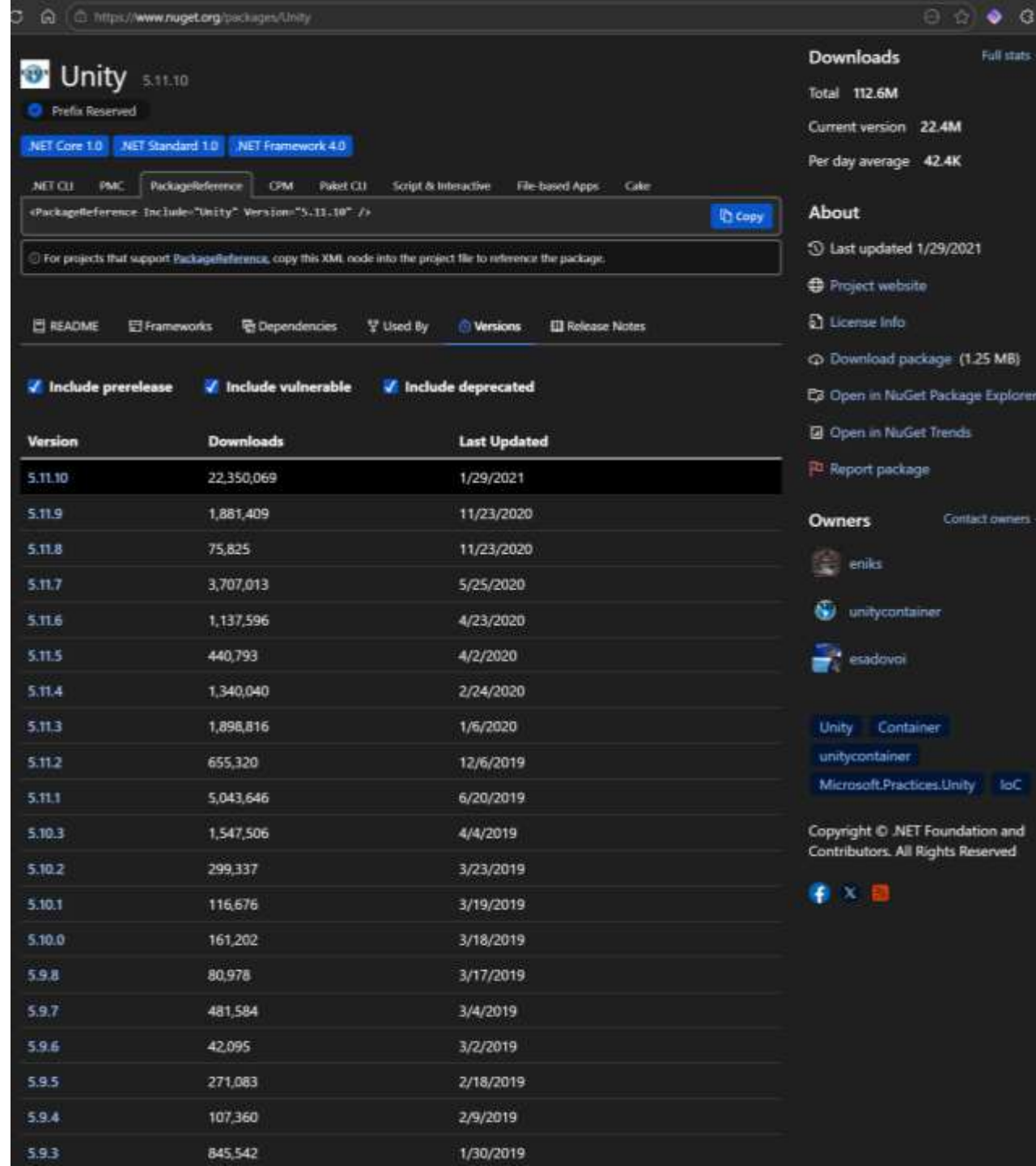
Microsoft.Practices.Unity IoC

Copyright © .NET Foundation and Contributors. All Rights Reserved

f x

Stuff you don't know you don't know IOC impact

- A decision had to be taken:
 - Refactor to something that is already in use but essentially dead / doubting if it's up to the task?
 - Bring in a new IOC container that is not known?



Unity 5.11.10

Prefix Reserved

.NET Core 1.0 .NET Standard 1.0 .NET Framework 4.0

.NET CLI PMC PackageReference CPM Paket CLI Script & Interactive File-based Apps Cake

<PackageReference Include="Unity" Version="5.11.10" />

Copy

For projects that support PackageReference, copy this XML node into the project file to reference the package.

README Frameworks Dependencies Used By Versions Release Notes

Include prerelease Include vulnerable Include deprecated

Version	Downloads	Last Updated
5.11.10	22,350,069	1/29/2021
5.11.9	1,881,409	11/23/2020
5.11.8	75,825	11/23/2020
5.11.7	3,707,013	5/25/2020
5.11.6	1,137,596	4/23/2020
5.11.5	440,793	4/2/2020
5.11.4	1,340,040	2/24/2020
5.11.3	1,898,816	1/6/2020
5.11.2	655,320	12/6/2019
5.11.1	5,043,646	6/20/2019
5.10.3	1,547,506	4/4/2019
5.10.2	299,337	3/23/2019
5.10.1	116,676	3/19/2019
5.10.0	161,202	3/18/2019
5.9.8	80,978	3/17/2019
5.9.7	481,584	3/4/2019
5.9.6	42,095	3/2/2019
5.9.5	271,083	2/18/2019
5.9.4	107,360	2/9/2019
5.9.3	845,542	1/30/2019

Downloads Full stats

Total 112.6M

Current version 22.4M

Per day average 42.4K

About

Last updated 1/29/2021

Project website

License info

Download package (1.25 MB)

Open in NuGet Package Explorer

Open in NuGet Trends

Report package

Owners Contact owners

eniks

unitycontainer

esadovoi

Unity Container

unitycontainer

Microsoft.Practices.Unity IoC

Copyright © .NET Foundation and Contributors. All Rights Reserved

f x

Stuff you don't know

You don't know

IOC impact

- ☛ Evaluated ServiceProvider / ServiceCollection
 - ☛ VASTLY different lifecycle
 - ☛ Unity - Wild wild west
 - add objects at any given time, BuildUp,...

```
[TestMethod]
new *
public void WildWildWest()
{
    var container = new UnityContainer();
    container.RegisterInstance(typeof(ISampleAddIn),
        new SampleAddIn(id: "First"));
    var resolved = container.Resolve<ISampleAddIn>();
    Assert.AreEqual("First", resolved.Id);

    container.RegisterInstance(typeof(ISampleAddIn),
        new SampleAddIn(id: "Second"));
    var resolvedAgain = container.Resolve<ISampleAddIn>();
    Assert.AreEqual("Second", resolvedAgain.Id); //WTF
}
```

Stuff you don't know

You don't know

IOC impact

☛ Evaluated ServiceProvider / ServiceCollection

☛ VASTLY different lifecycle

☛ Unity - Wild wild west

add objects at any given time, BuildUp,...

☛ ServiceCollection – Law & Order

Register types/instances on ServiceCollection

Create ServiceProvider from ServiceCollection

☛ That's it – changes to ServiceCollection will not come to already instantiated ServiceProvider

☛ This essentially breaks the usage patterns of UnityContainer – requiring HUGE breaking changes to the applications and to customer code

☛ Definitely a new major, any planned timeline is gone

```
[TestMethod]
public void LawAndOrder()
{
    var collection = new ServiceCollection();
    collection.AddSingleton<ISample, SampleA>();

    var provider = collection.BuildServiceProvider();
    var resolved = provider.GetRequiredService<ISample>();
    Assert.IsInstanceOfType(resolved, typeof(SampleA));

    collection.AddSingleton<ISample, SampleB>();

    var resolveAgain = provider.GetRequiredService<ISample>();
    //resolving from same instance does not see the new instance
    Assert.IsInstanceOfType(resolveAgain, typeof(SampleA));

    //Create new provider and resolve there
    var provider2 = collection.BuildServiceProvider();
    var resolved2 = provider2.GetRequiredService<ISample>();
    //should return B as it was added last
    Assert.IsInstanceOfType(resolved2, typeof(SampleB));
}
```

Stuff you don't know

You don't know

IOC impact

- A decision had to be taken (again):
 - Refactor to something that is already in use but essentially dead / doubting if it's up to the task?
 - Bring in **an additional** IOC container that is somewhat known – while leaving the existing usage on the old one?

```
[TestMethod]
Klaus Linzner
public void MultiInject()
{
    var serviceCollection = new ServiceCollection();
    serviceCollection.AddSingleton<IMultiService, MultiServiceA>();
    serviceCollection.AddSingleton<IMultiService, MultiServiceB>();

    var provider = serviceCollection.BuildServiceProvider();

    var x:IMultiService[] = provider.GetServices<IMultiService>().ToArray();
    Assert.AreEqual(2, x.Length);
    foreach (var elem:IMultiService in x)
    {
        Console.WriteLine(elem.GetType());
    }
}
```

Stuff you don't know

You don't know

IOC impact

- More of a gut-feeling decision:
You know it should be done
you know it will hurt
- Pretty much went all-in on IOC
 - Tried to get rid of implementations in all static convenience classes
 - Created extension methods instead
 - Static classes use extension methods on compat layer
 - Only an extremely thin layer survived
 - Small compat layer for those (RuntimeServices)
 - Use IOC / clear(er) lifetime for everything else

```
public static class RuntimeServices
{
    private static IServiceProvider serviceProvider;
    private static bool alreadySetup = false;

    Klaus Linzner
    internal static void Reset(){...}

    8 56 usages Klaus Linzner
    public static IServiceCollection Initialize(){...}

    8 5 usages Klaus Linzner
    public static bool TryGetService<T>(out T t)
    where T: class {...}

    8 5 usages Klaus Linzner
    public static bool TryGetServiceProvider(out IServiceProvider existingServ

    8 62 usages Klaus Linzner
    public static IServiceProvider ServiceProvider {...}

    8 2 usages Klaus Linzner
    private static void SetServiceProviderDuringRuntimeStart(IServiceProvider

    Klaus Linzner
    public static IServiceProvider UseIOC(this IServiceProvider provider){...}
    8 22 usages Klaus Linzner
    public static IServiceProvider UseIOC(this IServiceCollection collection){
}
```

Stuff you don't know

You don't know

IOC impact

- Awesome for general architecture
 - Finally giving full control to customer projects – essentially finishing what we started with the updated Release mechanism
 - They can change entire surrounding – from configuration to logging infrastructure to implementations to web endpoints without PD needing to provide extension points.

```
class Program
{
    /// Main entry-point for this application. ...
    // Klaus Linzner +1
    static async Task Main()
    {
        await RuntimeServices.Initialize()
            .AddIOCAApplicationDescription<Program>(ConfigurationSettings.Application)
            .AddIOCRuntime()
            .AddIOCUnityContainer()
            .AddIOCServerDefaults()
            .AddIOCSysmonMonitor(provider: serviceProvider => new ResourceManagerTranslator(
            .AddIOCMetrics()
            .AddIOCORacleProvider()
            .AddIOCCASPNetCore(7036)
            .AddIOCCASPNetCore_OData()
            .AddIOCCASPNetCore_ODataService<DeviceGatewayODDataServiceDefinition>()

#if NET48
            .AddIOCRfc1006()
            .AddIOCMalfunctionMonitor() // IServiceCollection
#endif

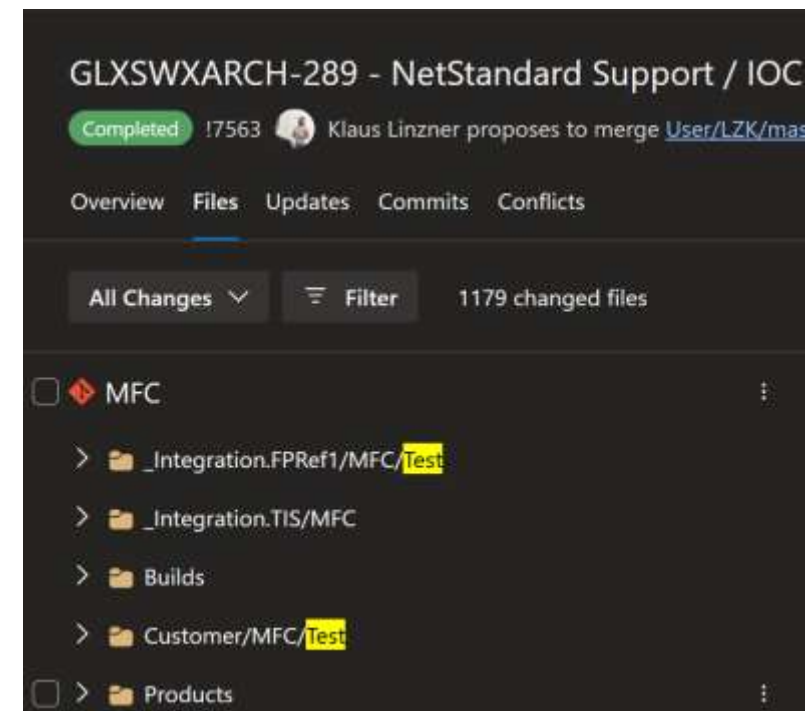
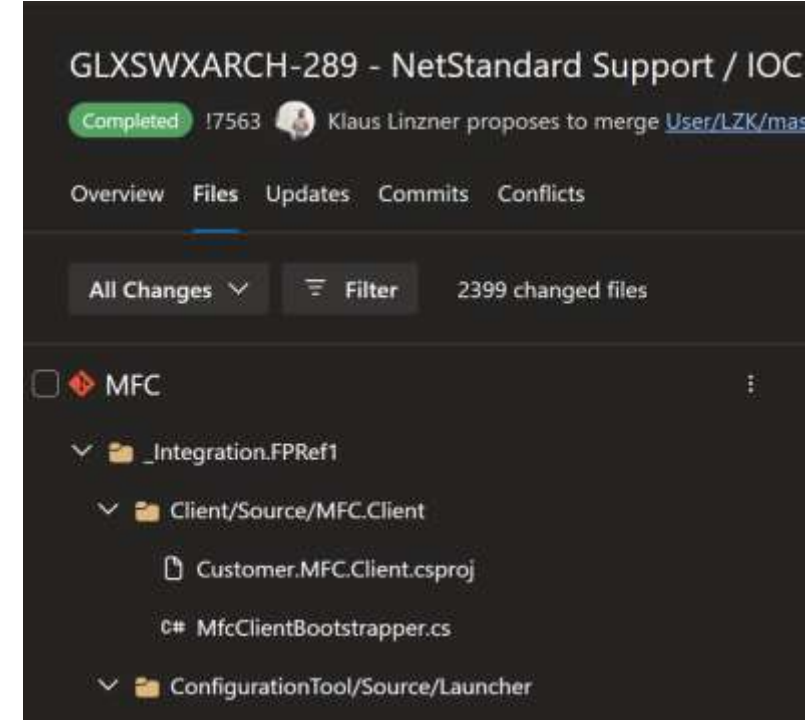
        .RunAsync<CustomDeviceGatewayApplicationLoader>(); // Task
    }
}
```

Stuff you don't know

You don't know

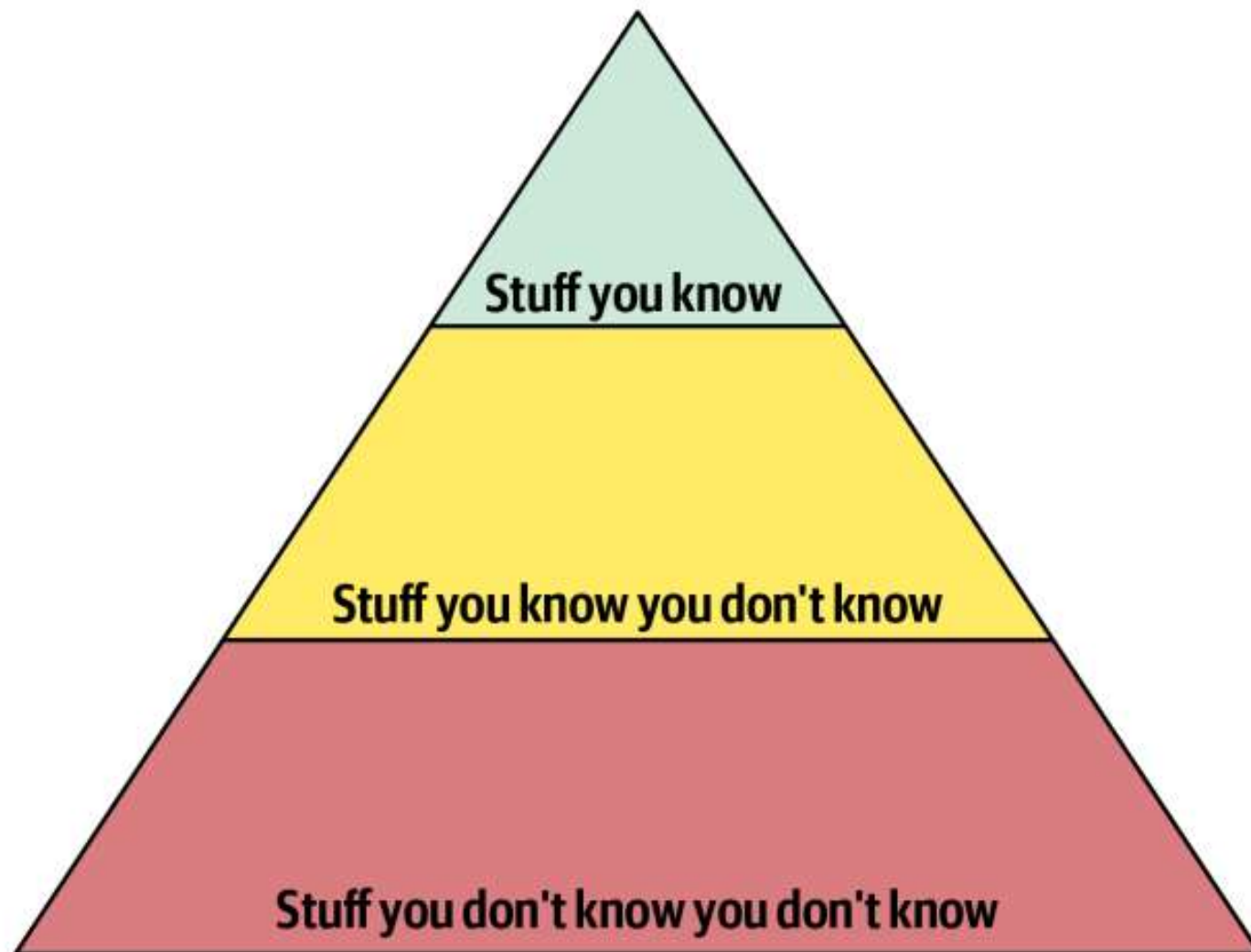
IOC impact

- Devastating when taking effort and implications into account
- Almost half of all changes were test files that needed to be updated because of it



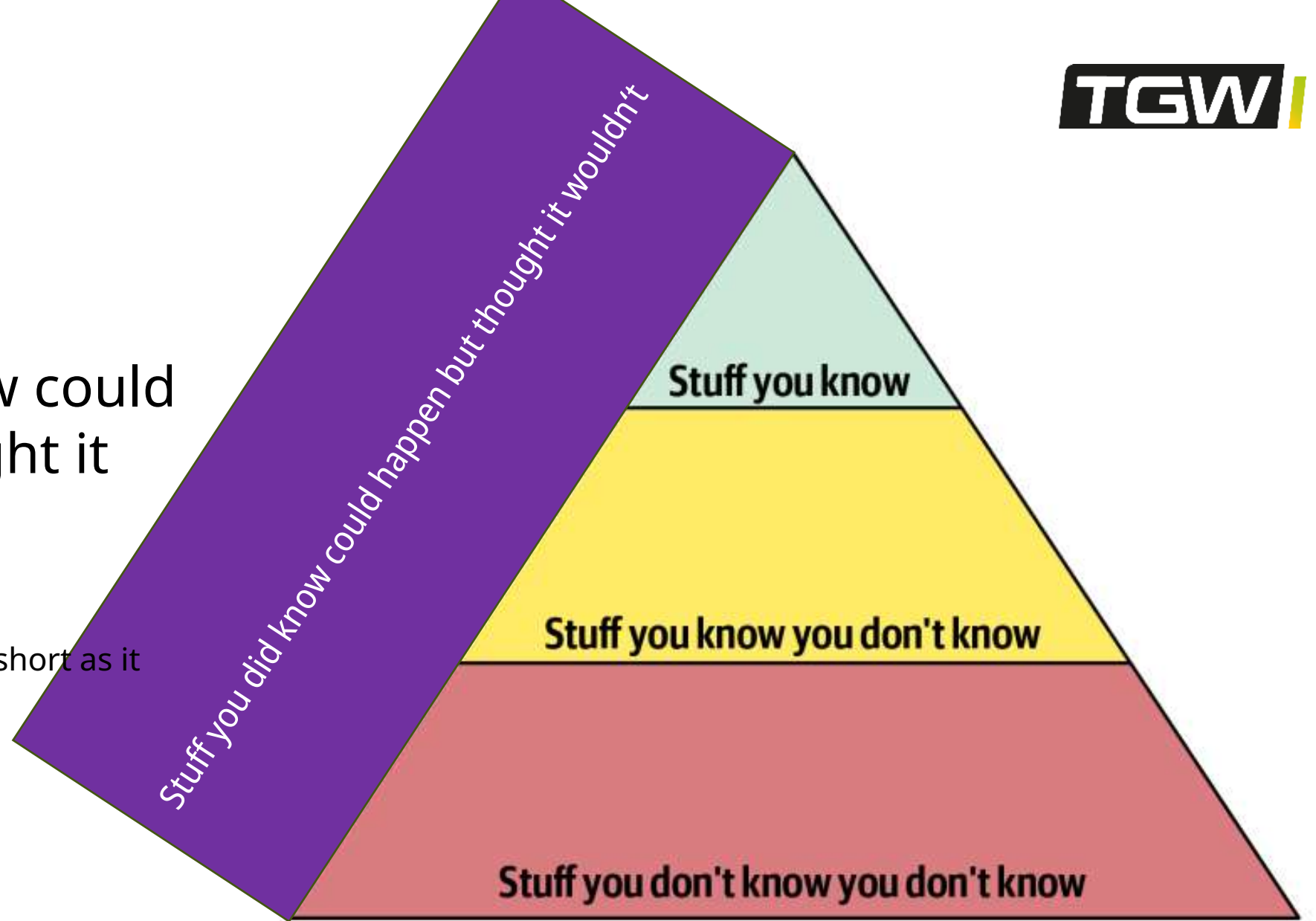
This one is slightly wrong

Introducing...

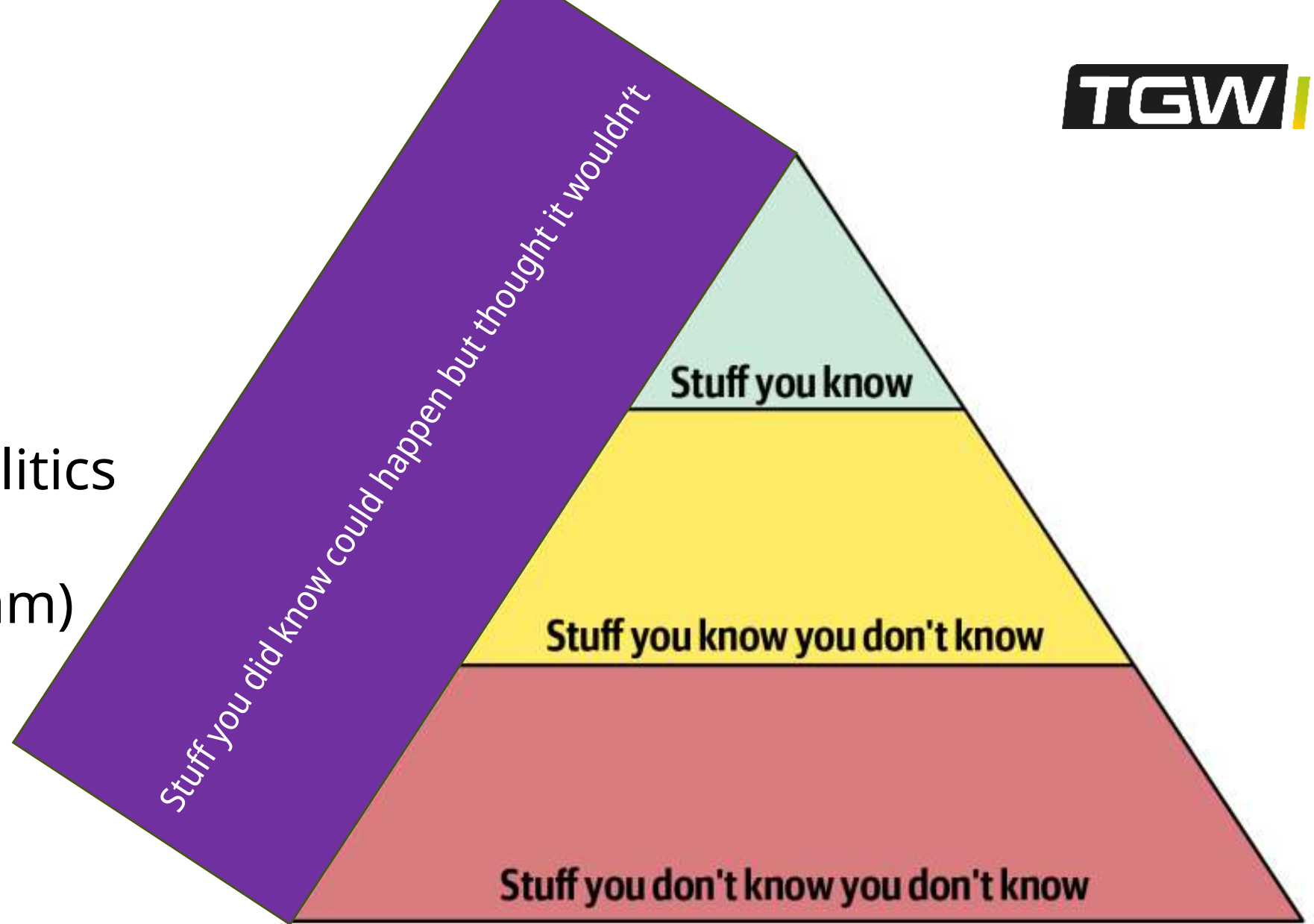


Stuff you did know could happen but thought it wouldn't

(don't worry – this section is as short as it was painful)

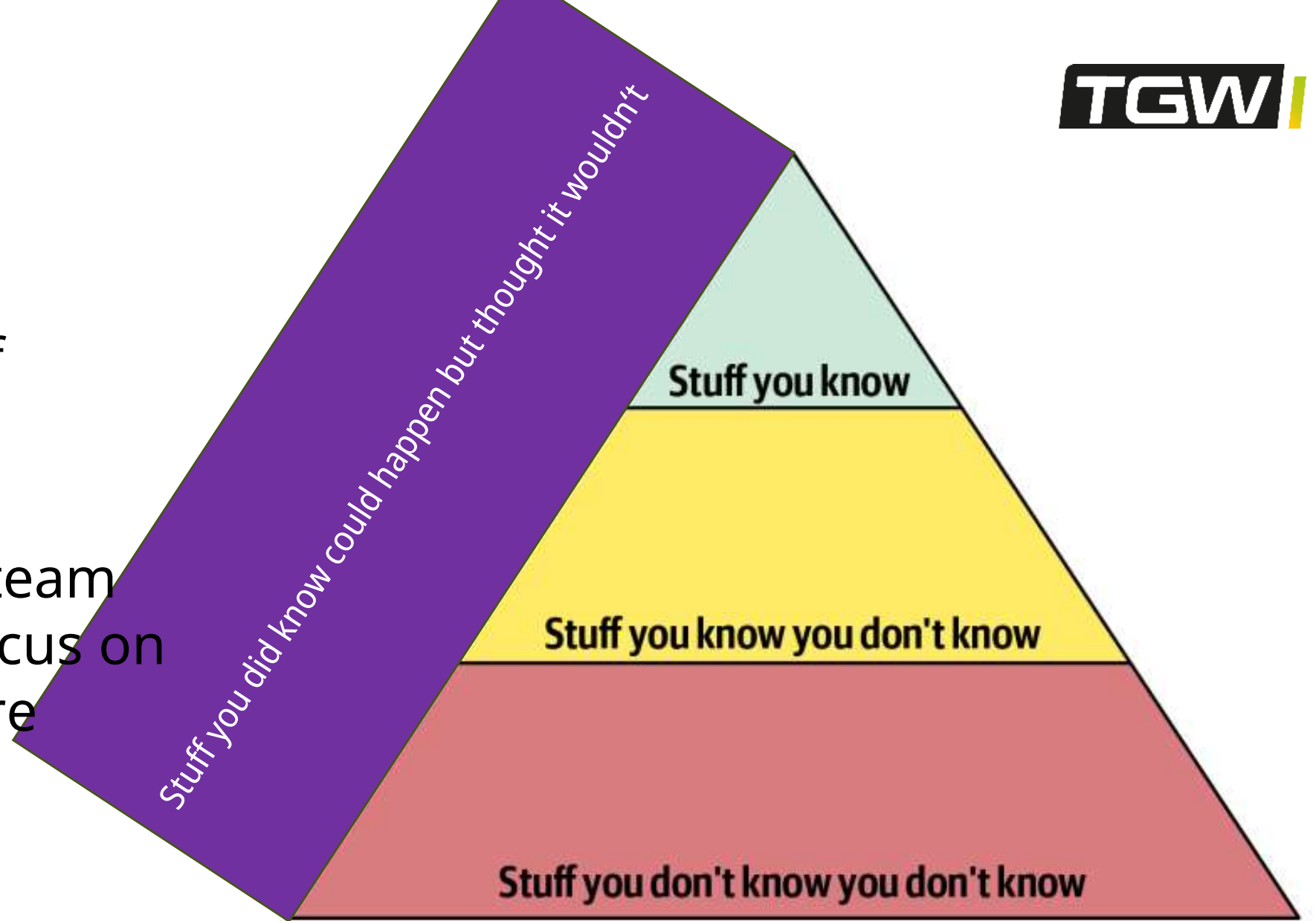


The #1: (office) politics
(not in the dev team)



#2: we sold lots of projects

To the point that team could not work/focus on migration anymore



#3: MERGE HELL INDUCED BY DELAY FROM POLITICS & PROJECTS

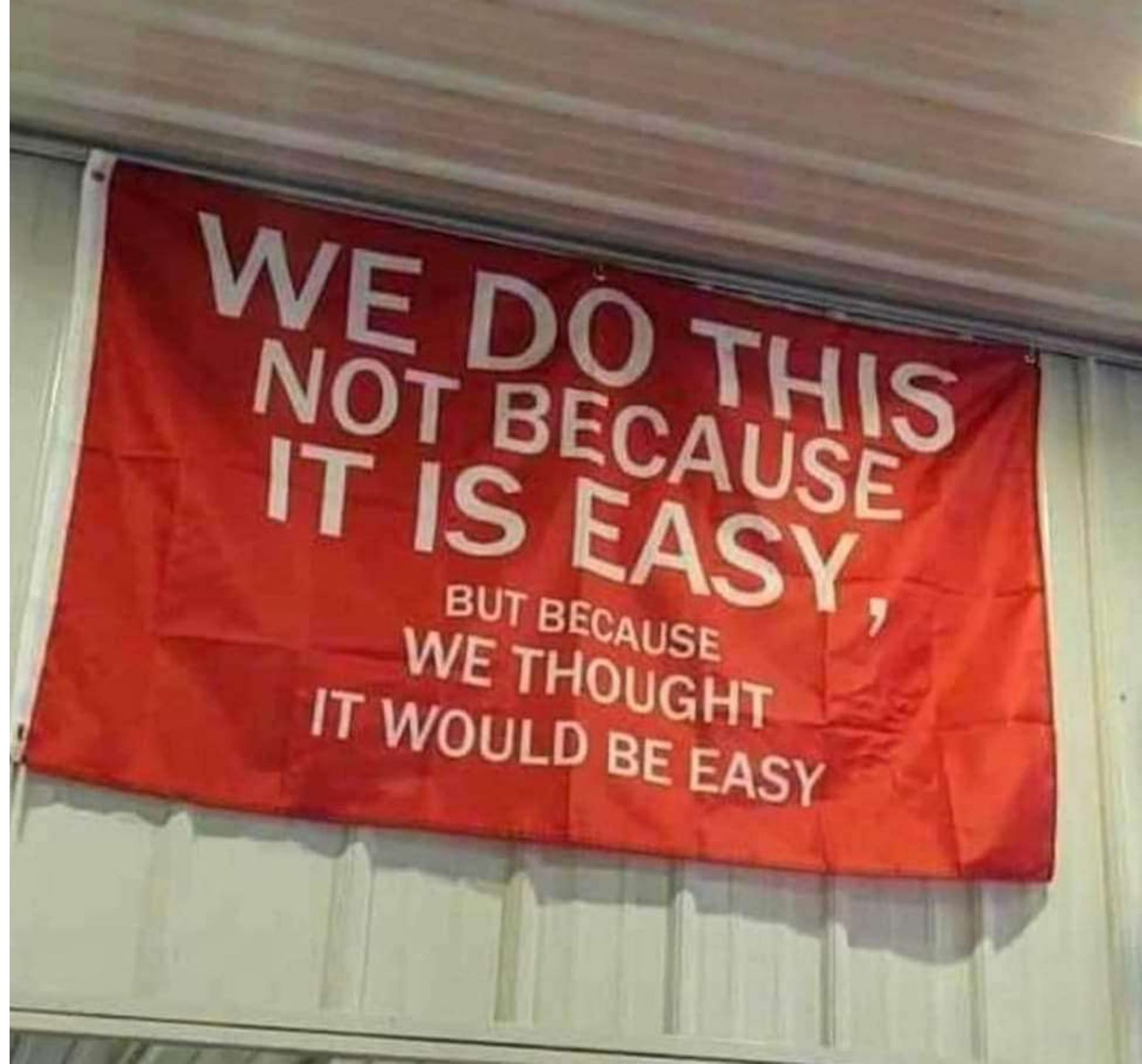
**20 DEVS DEVELOPING AND
SHIPPING FEATURES
VS
1 DEV MIGRATING AND
MERGING CHANGES**

A wide-angle photograph of a modern industrial warehouse or factory floor. The space is filled with complex machinery, including multiple levels of conveyor belts and automated sorting systems. In the center-right, a worker wearing a black t-shirt with a white 'CI' logo and blue jeans stands near a stack of cardboard boxes. The floor is light-colored, and the ceiling is high with numerous fluorescent lights. A large, semi-transparent red shape is overlaid on the left side of the image, containing the text 'Wrap up'.

Wrap up

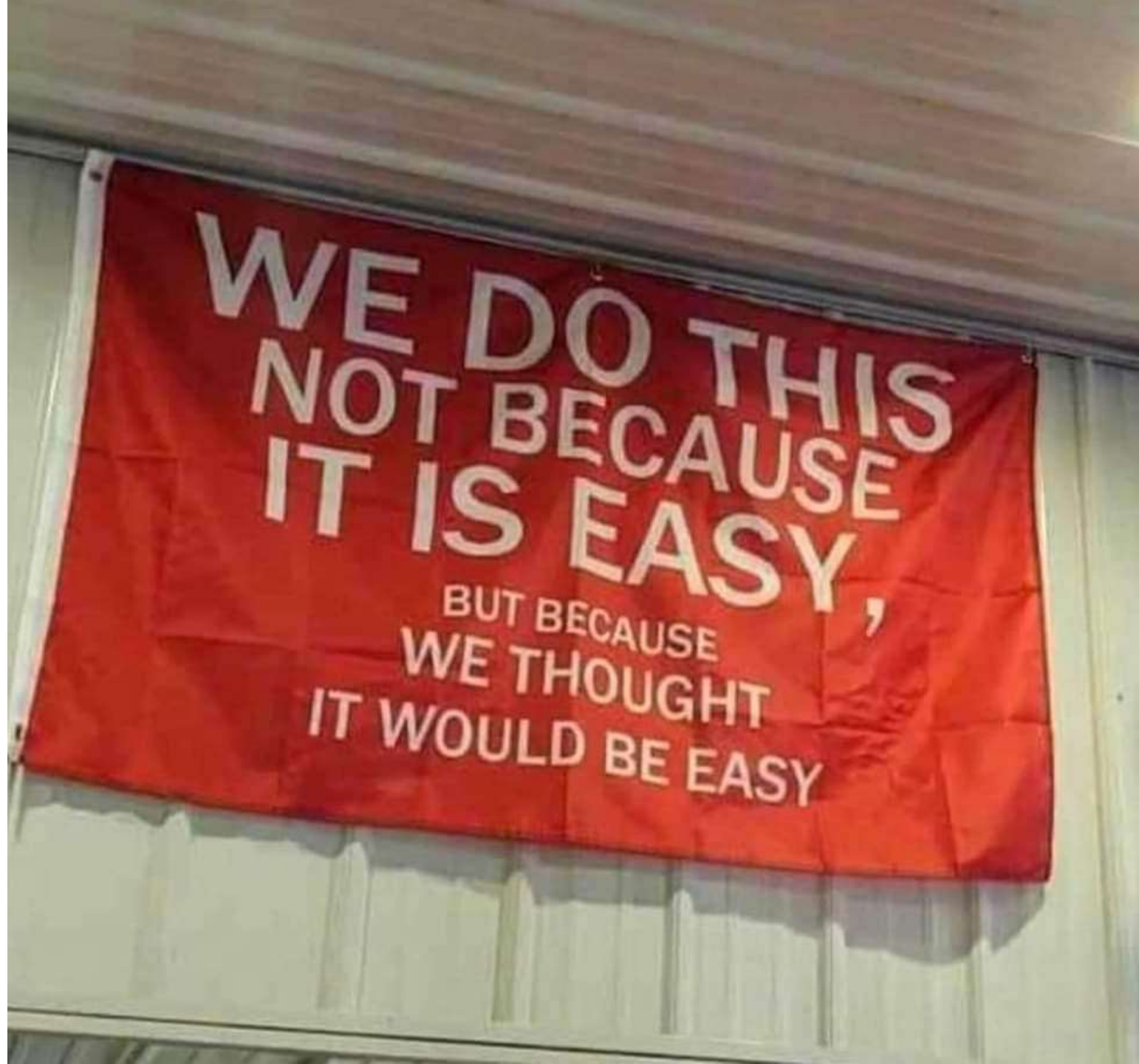
Rough timeline

- 2021
 - Decision onto containerization around october, staffing & planning starts.
 - .NET core migration not a project on its own but part of a bigger project „containerize everything“
- 2022
 - Evaluation of platform (Red Hat Openshift, VMWare Tanzu, Hashicorp Nomad)
 - Technial evaluation was the fun / interesting part
 - Politics involved was beyond insane
 - First technical pre-requisites on .NET Code
- 2023
 - Design and structure of gitops workflows
 - Re-Evaluation - little technical development on .NET migration (tech spending time in politics)
- 2024
 - First half: Finished heavy lifting of .NET migration
 - Second half:
 - First installation at customer
 - Massive internal trainings (worldwide)
- 2025
 - Rollout starting (multiple installations)



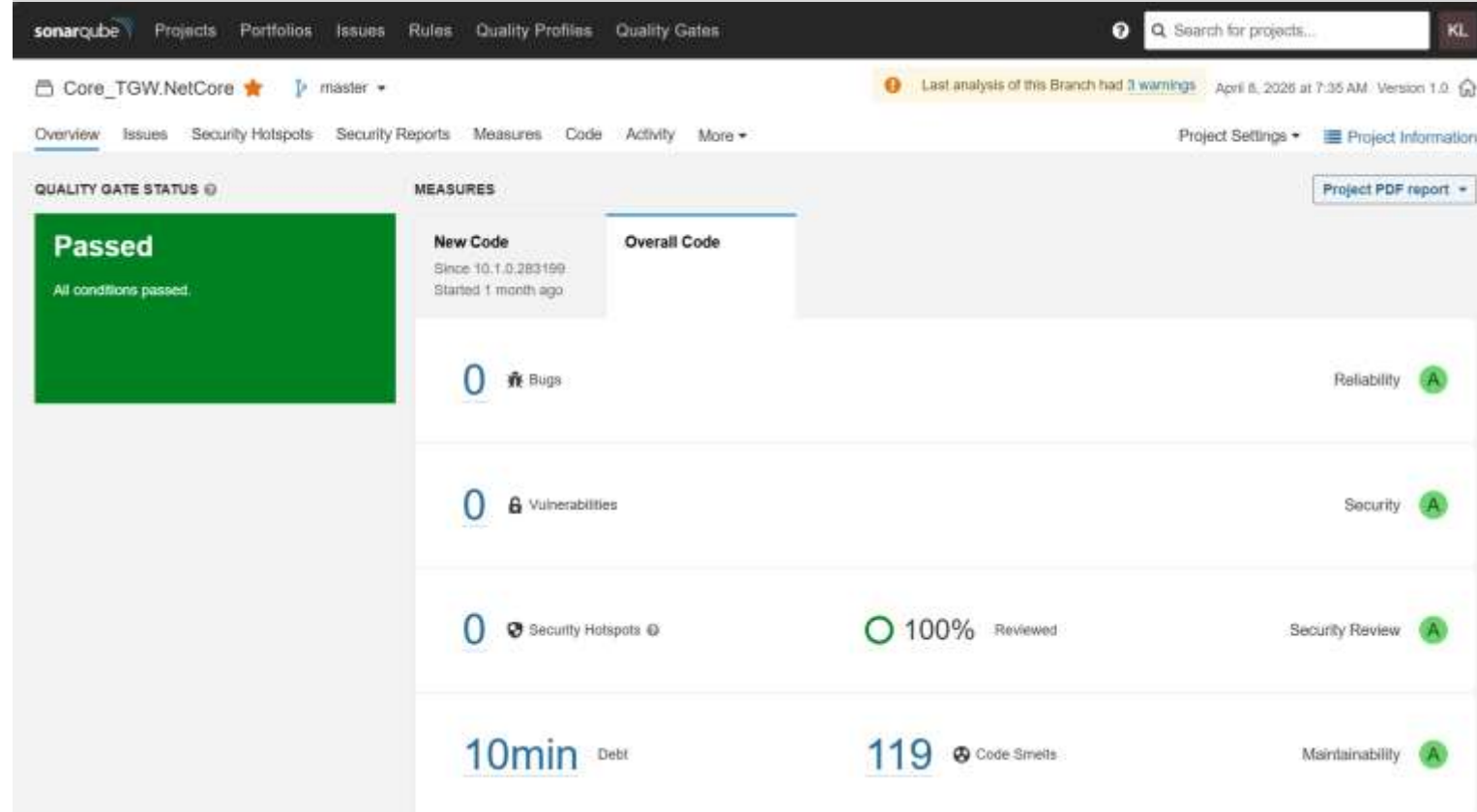
Lessons learned

- Preparation on the actual required stuff was OK
- As we focussed on Server/backend and ditched WPF frontend, vast majority went smoother than expected
- Stumbled over „Screw this – let’s change from Unity to general IOC in a clean way“
 - Migration effort doubled with this single change (that was not strictly necessary)
 - In hindsight: some shortcuts could have been taken
 - Vastly underestimated impact on test cases
 - Partially: Made small concise changes to bring back harder -> merge hell
 - Longer Term benefit, but migration was...crunchy



Lessons learned

- Soon after completion
 - Fork / copy of the repo – moved cross functional base libraries out of the application repo to shrink them / clean them and make them easily available to other teams
 - Made easier to give clean path / guide to all other .NET teams that does not suffer from 20 year old decisions / force them into old patterns



Lessons learned

- Soon after completion
 - IOC change was the most painful - but right thing to do
 - Great foundation to provide capabilities in an easier way

eComponent.Hybrid\Program.cs

```
#region
```

```
await RuntimeServices.Initialize()
.....AddIOCAppliationDescription( applicationname: "HybridService", version:
.....AddIOCRuntime()
.....AddIOCServerDefaults()
.....AddIOCCASPNetCore()
.....AddIOCOpenTelemetry()
.....//This component acts both as server and client - it hosts the hybr
.....//It's possible to have both facade for one service and agent for an
......AddIOCPProxyAgent<ISampleService>( myPartyId: "HybridService", serve
......AddIOCPProxyFacade<IHybridService, HybridService>( "HybridService"
.....//If we're using the same connection for agent and facade - and we'r
.....//otherwise we could deadlock ourselves: while receiving the message
.....//please keep in mind: empty sequencing array means that the message
......AddIOCPProxyFacadeOffloading<IHybridService, Correlated<StartAnother
......AddSingleton<HybridService>()
......AddIOCDefaultApplicationLoader() // IServiceCollection
......RunAsync(); // Task
```

Three large, overlapping, leaf-like shapes in red, yellow, and green. The yellow shape is in the center and contains the text "Thank you". The red shape is to the left, and the green shape is to the right. They all have a wavy, organic edge.

**Thank
you**